

Distributed Systems

Chun-Feng Liao

廖峻鋒

Department of Computer Science

National Chengchi University

有關分組 (3/4 due)

- 要件
 - 每組6-8人
 - 上傳期限
- 找不到組員？
 - 可使用FB群組
 - 下課時間
- 一人代表上傳分組名單
 - 每個人的姓名、系級、學號

Distributed Systems

Comprehending Software Systems

Chun-Feng Liao

廖峻鋒

Dept. of Computer Science

National Chengchi University

解讀大型系統

- 解讀素材
 - 原始碼/原始碼註解
 - 文件
 - 描述該技術的專書
 - 網路資訊 (blogs/articles/論壇)
 - 資料基礎訓練
 - 資料結構/演算法
 - 物件導向
 - 計算機結構/作業系統
 - 網路通訊

解讀大型系統

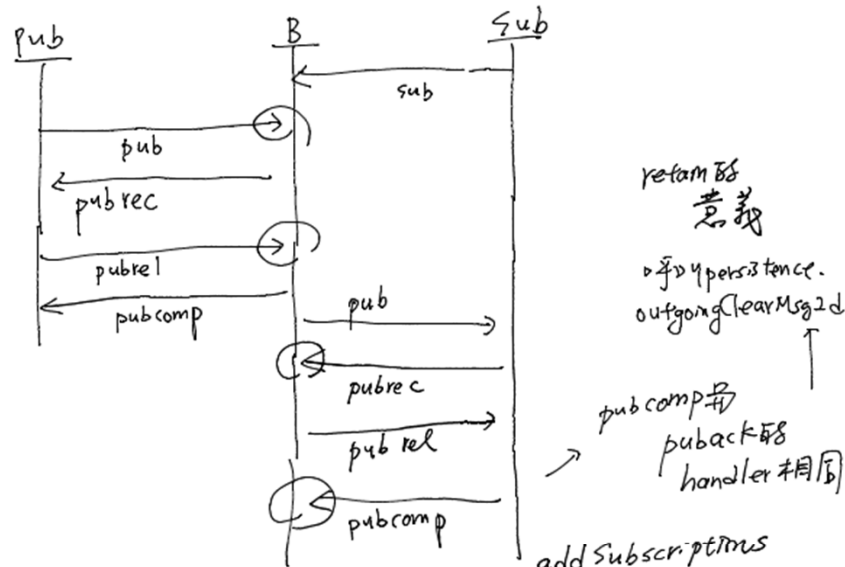
- 功能解析
 - 了解該軟體的目標
 - 了解重要功能、製作筆記、圖表
 - 問問題: 用問題來導引你的探索
 - “為何要有它?”、“可以用做什麼?”、“怎麼用?”、“特色在那”、“什麼情況適合用它?”

解讀大型系統

- 實現解析 (反向工程) *Reverse Engineering*
 - 這個軟體是如何實現上述功能的?
 - 不斷提問，透過素材來「推測」與「驗證」你的想法
 - 配合圖表
 - 初步解析時，不要太拘於型式，以任何你覺得看得懂的形式都可以
 - 之後可進一步整理為: 類別圖、循序圖、活動圖
 - 文字說明紀錄與圖同等重要

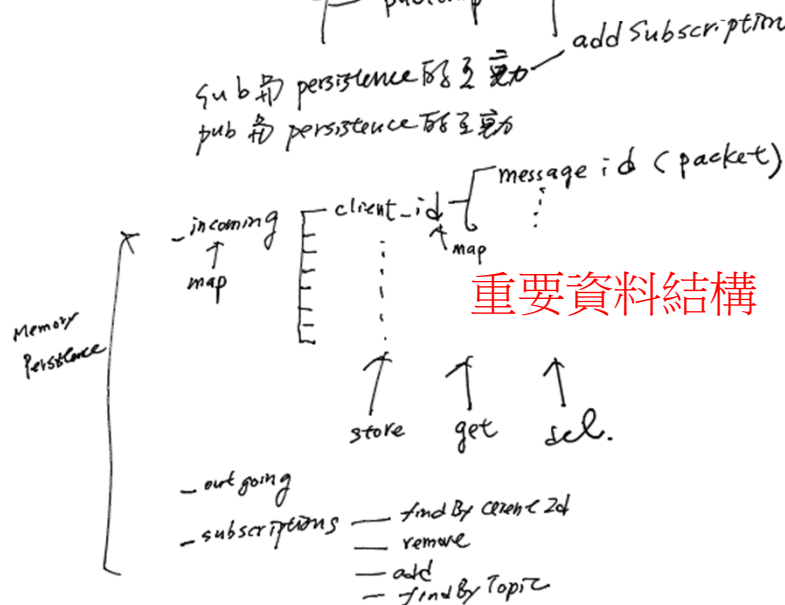
Case: MQTT Broker

Qos 2 背景知識



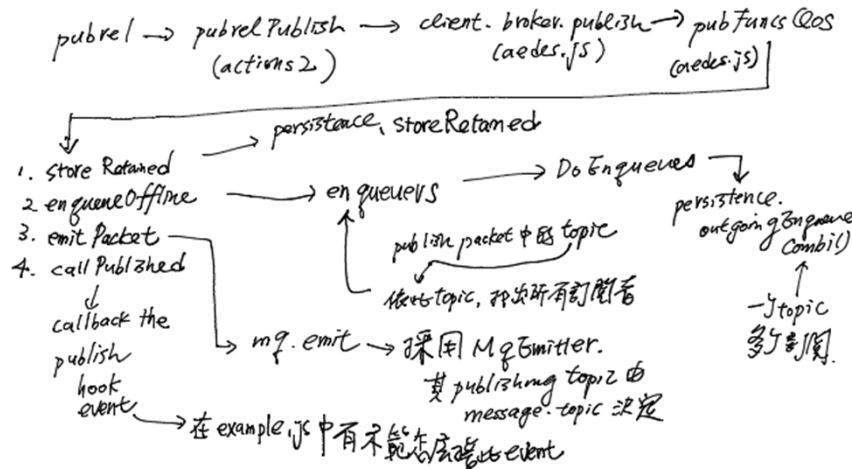
M essage
Q ueueing
T elemetry
T ransport

⇒ MoM



Aedes-Persistence
Incoming
Outgoing
Subscriptions
Will
Retained

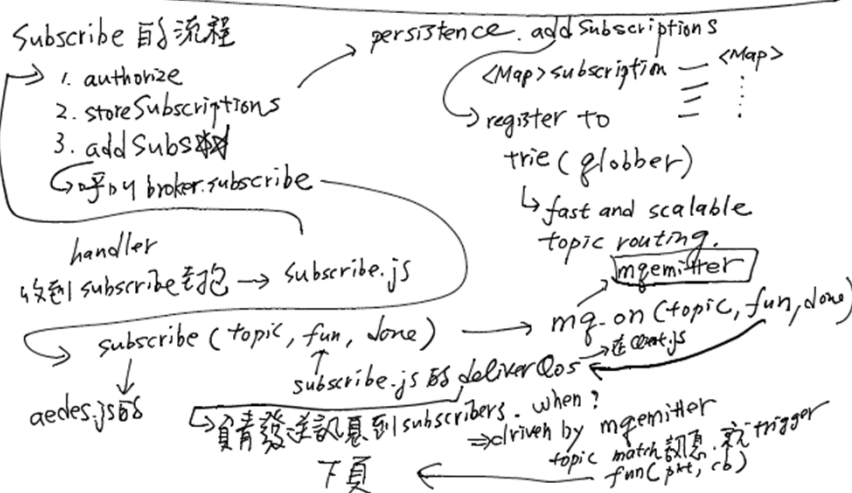
初步解析



Subscriber 從 mq 中取到訊息. 流程為何?

↓
Broker 發送訊息給 subscriber 的 "publish" 是怎樣的建立?

(client.js) deliver @os



初步解析通常會充滿疑問、容易迷失
「功能」就是你的導航器，聚焦在你要了解的問題
不斷提問，在程式碼中驗證你的猜測

mqemitter 如何得知要傳入 (packet, cb)?

mcollina/mqemitter.

emitter.on(topic, listener, cb)

listener 的 signature: fun(message, done)

語意: 當 topic 收到特定訊息, 就呼叫 listener.

呼叫完後, 以 cb 結尾.

所以上
client.deliver @os 方法就是 match topic 時必定會呼叫的 cb.

deliver @os 中, 做了些什麼

決定是否立即送訊息

是 → write @os → write.js → to subscribers
否 → persistence.outgoingUpdate

setImmediate 的意義?

write @os

Retain 和 Clean 的意義

寫程式測 mqemitter.

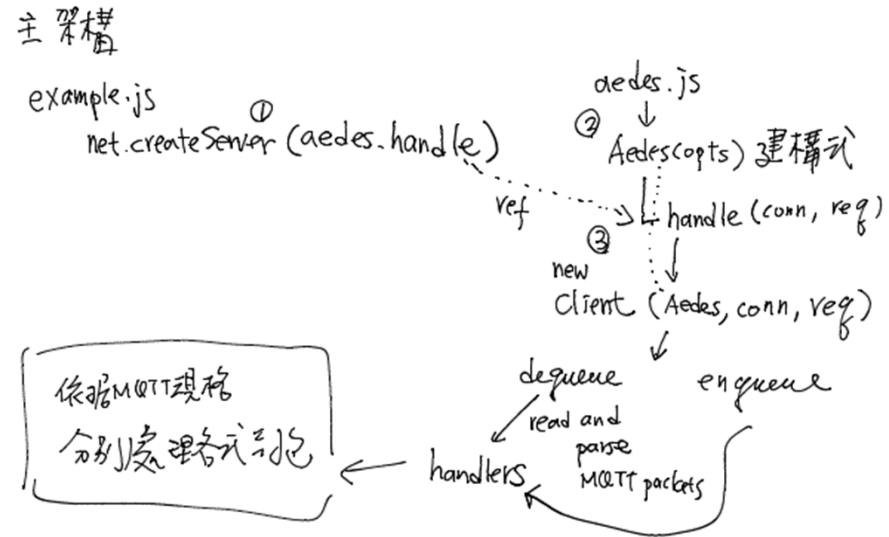
? setImmediate:

Node.js Event Loop:

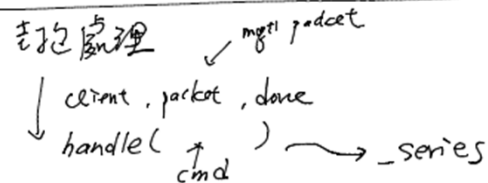
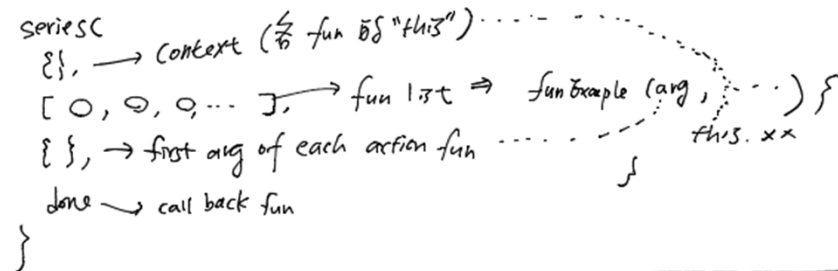
- 1 I/O event
- 2 setImmediate
- 3 set Interval
- 4 process.next tick

歸納解析

隨著對專案結構與程式碼理解的加深，在一定程度初步解析後，可以開始歸納出比較高層次、有系統的理解圖示



_series 結構



針對之前的提問，可以有比較詳細的理解與說明

有些疑問在官方文件或程式註解中可找到答案

需要記錄解析的結構：

Subscription (subscribe.js)：其中的 fun 就是 handler fun
可能是 deliver, deliverQoS

此 Fun 本會透過訂閱程序註冊在 mgemitter 中。因此 match 時才能將 deliver, 寄出給 clients.

④ handler 的 actions

persistence

找出 publish 的 sequences

raw packets
conn

nextBatch

parse

parser

on packet

Engine

parser queue

once connected

queue

handle(modules)

conn.on('readable', nextBatch)

buf ← conn.read

subscribe 的 4 個 actions: addSub (subscribe.js)

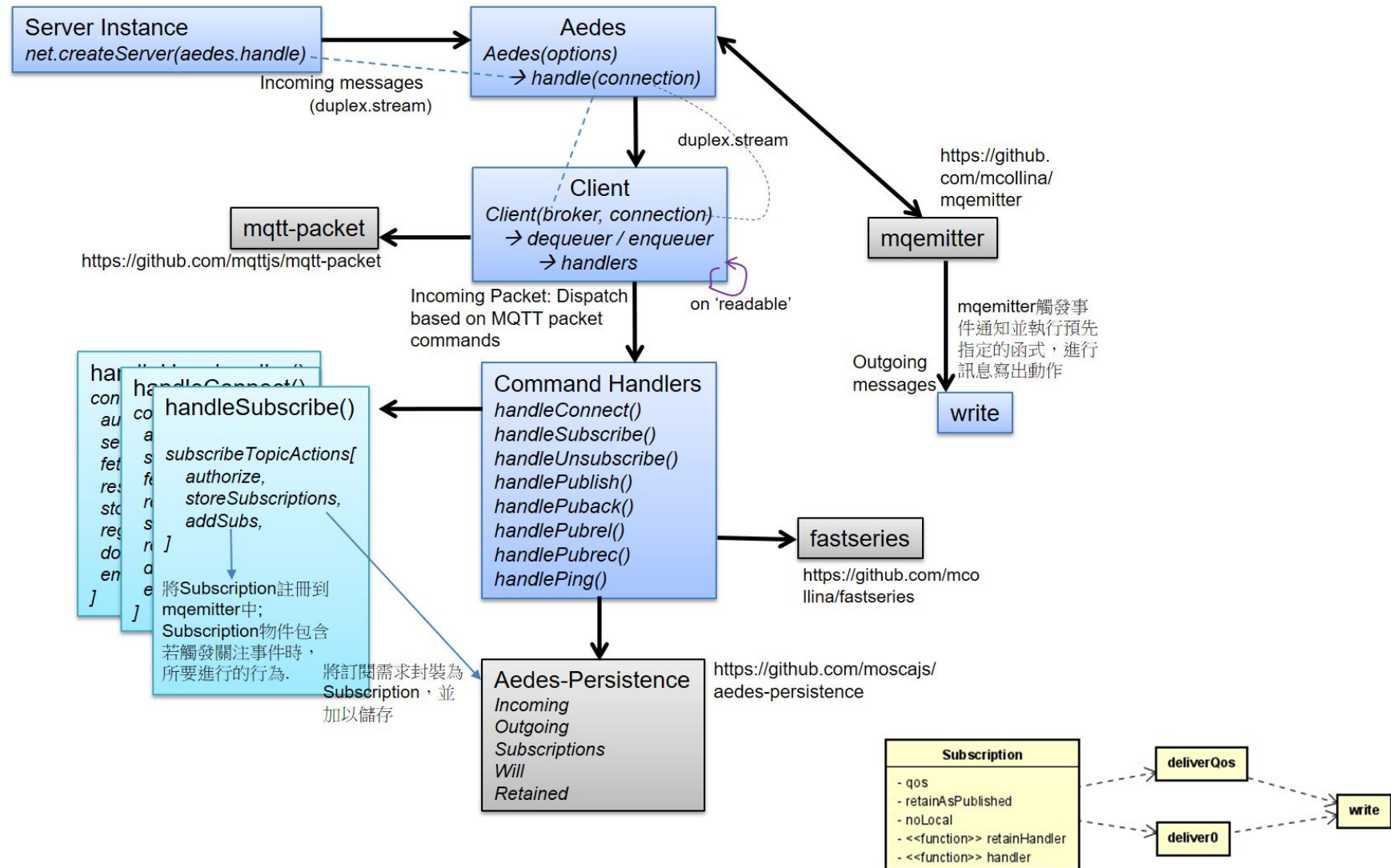
by broker, subscribe

mg.on(topic, fun, done)

deliver or deliverQoS
會先打已取
handle Packet Subscription

在官方文件中：
readable means
new data is
available.

描述「你所理解」的系統



解析報告

4.1 Aedes 開源 MQTT Broker 解析與修改

為評估可行性，我們對所設計的機制分別實現在 MQTT 端點與 Broker 上，並實際佈署到區域網路與嵌入式硬體及感測節點。硬體方面，我們希望藉由計畫經費所購置的 Raspberry Pi (版本: 4 B) 嵌入板來實現 MQTT Broker，而使用 Arduino WiFi (或 What's next) 實現功能精簡的 MQTT 端點。軟體方面，如同前面提到，我們基於開放源碼的 Aedes 專案 (<https://github.com/moscajs/aedes>)，並修改程式碼來完成上一章所設計的端點間確保送達機制。

首先，根據研究過程所得到的經驗，我們對整個 Aedes 專案的原始碼架構做一個簡單的介紹與解析，如圖 3。Aedes 採用 Node.js 內建的 net 套件做為網路通訊的基礎架構，所以一開始是採用 net.createServer 來寫作網路伺服器，在連線建立後，便透過 aedes.handle 傳入 Aedes 的核心。Aedes 主要進入點為 handle 方法，連線建立後，Node.js 會傳入一個已建構的 Socket 類別實體進 handle 方法做為參數，也具有 duplex.stream 特性。邏輯上一個連線可視為一個 Client 實體。連線收到資料後 (duplex.stream 的 readable 事件被觸發)，會透過 mqtt-packet 專案⁶解碼。

```
// in client.js
conn.on('readable', nextBatch);
function nextBatch(err) {
  // ...
  if (that._parsingBatch <= 0) {
    that._parsingBatch = 0;
    const buf = client.conn.read(null); // read next packet
    //console.log(buf);
    if (buf) {
      client._parser.parse(buf); // decode via mqtt-packet parser
    }
  }
}
```

解碼後，parser 會觸發 packet 事件，並被 enqueue 函式處理，該函式會依據其內含之 MQTT 命令 (如: connect、subscribe、publish、puback、pubrel、ping 等) 委交給相對應的 handler 做完後，會再呼叫 nextBatch 讀入下一批封包。由下面的第一行可以知道 handle 的具體實作是在 ./handlers/index.js 中。

```
// in client.js
const handle = require('./handlers');
// ...
this._parser.on('packet', enqueue);
// ...
function enqueue(packet) {
  // ...
  // already connected or it's the first packet
  if (client.connackSent || client._parsingBatch === 1) {
    handle(client, packet, client._nextBatch);
  } else {
    // ...
  }
}
```

⁶<https://github.com/mqttjs/mqtt-packet>

在原始碼的 handlers 目錄中，相對應的指令處理邏輯是分別寫在不同的檔案中，例如 pubrel 的處理邏輯就在 handlers/pubrel.js 中。由下面的程式碼片段中，可以看出 handle 會指派給不同的 handle 函式處理，這些函式是自不同檔案匯入。

```
// in ./handlers/index.js
const handleConnect = require('./connect')
const handleSubscribe = require('./subscribe')
const handleUnsubscribe = require('./unsubscribe')
const handlePublish = require('./publish')
// more imports...
function handle (client, packet, done) {
  // ...
  switch (packet.cmd) {
    case 'publish':
      handlePublish(client, packet, done);
      break;
    case 'subscribe':
      handleSubscribe(client, packet, false, done)
      break;
    case 'unsubscribe':
      handleUnsubscribe(client, packet, done)
      break;
    // ...more cases
    // ...other code
  }
}
```

為了能處理斷線重連、封包重送等永續性機制，需要一個處所來暫時存放收到的封包，在 Aedes 中是另外實作了一個專屬的簡易記憶體資料庫，稱為 Aedes-Persistence⁷，它也提供了 mongodb、redis、leveldb 和 neDB 的連接口，以儲存在這些資料庫中。如圖 3 所示，Aedes-Persistence 所保存的資料包含了 Incoming(外部傳入封包)、Outgoing(暫時保存，準備送出的封包)、Subscriptions(訂閱註冊)、Will(失效節點預留的後處理)和 Retained(斷線重連的支援)。從這裡我們可以找到第一個實作的切入點，在程式碼 1 和程式碼 2 中的 outbox 概念，就需要在此和原系統做整合。

Aedes 的 Topic 對應與訊息遞送匹配、通知模型核心採用了另一個開源專案：MQEmitter⁸，MQEmitter 事實上也是專為 MQTT 設計，內部採用了許多優化來增進效能。它的 API 和 Node.js 的 Emitter API 類似，要使用它，首先要拿到 emitter 實例，emitter 支援平行處理，因此可以透過 concurrency 數量來指示。它的慣例是必須在訊息中指定 topic，如此 mqemitter 才知道要送到那個目標，之後，若要訂閱，則使用 on，並在其參數代入 topic name 即可，如下所示。

```
const mq = require('mqemitter');
const emitter = mq({ concurrency: 5 });
const message = { topic: 'mytopic', payload: 'or any other fields' };
emitter.emit(message, function () {
  // callback
})

emitter.on('mytopic', function (message, cb) {
  // message processing logics
  cb(); // callback when done
});
```

MQTT 主要有兩類的端點：訂閱者 (Publisher) 和接收者 (Subscriber)。Subscriber 會透過上述機制連線並送入帶有 SUBSCRIBE 指令的 MQTT 封包，如前所述，依據指令，會被委派給 subscribe.js 處理，此時 subscribe 的處理切分為多個步驟，每個步驟以一個函式表示，它們會依序執行。為講求效能，Aedes 的「依序」和「平行」都經過特殊優化機制，因此另外實作了特別的高階結構模組 (Higher-Order Control Structure, HOCS)，稱為 fastseries⁹與 fastparallel¹⁰。HOCS 具有固定的使用模式，開發人員需依指定的方式予以呼叫，就可以讓傳並其中的工作得到優化，以 fastseries 為例，下面的寫法可以依序執行 f1, f2 與 f3，且在這三個函式，第一個傳入的參數在 series 中指派，其 this 也會被改寫，整批工作執行完後，會呼叫 done 這個 callback 函式。fastparallel 具有相當類似的功能，在此就不多加說明。

```
const series = require('fastseries')();
series(
  {something}, // the context of functions, can be obtained using "this."
  [f1, f2, f3], // functions to call
  {arg1:"value1", arg2:"value2"}, // passed arguments, will appear in the
    first argument of the functions
  done // the function to be called when the series ends
)
```

以 subscribe 來說，它就是使用了 fastseries，藉以依序執行了三個步驟：authorize, storeSubscriptions 與 addSubs 等三個工作 (函式)。其中，addSubs 會產生新的 Subscription 物件 (如圖 4)，該物件的建構式參數中就包含了 handler 函式，登錄了訊息處理的行為。Subscription 物件產生後，會存在 client 的 subscriptions 物件中，之後，會呼叫 broker.subscribe，將訊息處理函式 (依據 QoS 設定不同，可能為 deliverQos 或 deliver0) 也登錄在 mqemitter 中。

```
// in subscribe.js
client.subscriptions[topic] = new Subscription(qos, func, rh, rap, nl)
```

⁷<https://github.com/moscajs/aedes-persistence>

⁸<https://github.com/mcollina/mqemitter>

⁹<https://github.com/mcollina/fastseries>

¹⁰<https://github.com/mcollina/fastparallel>

```

broker.subscribe(topic, func, done)
// in aedes.js
...
Aedes.prototype.subscribe = function (topic, func, done) {
  this.mq.on(topic, func, done)
}

```

在 subscriber 完成訂閱後，接下來就要等待 Publisher 進行訊息發佈。Publisher 會發出 MQTT 的 PUBLISH 命令，broker 收到後，依據前面提過的分派機制，會由 handlePublish(client, packet, done)，交給 publish.js 處理。在 Aedes 中，PUBLISH、PUBACK 與 PUBREL 是放在同一個檔案 (publish.js) 中處理。在其 enqueuePublish 函式中，會依據 QoS 進行不同的處理，QoS0 就直接呼叫 broker 的 publish，QoS1 與 2 則需要結合 PubAck 和 PubRel 來做接下來的互動：

```

// in publish.js
function enqueuePublish (packet, done) {
  const client = this
  switch (packet.qos) {
    case 2:
      // 存入處理函式，未來packet清掉時，就會觸發送出
      client.broker.persistence.incomingStorePacket(client, packet, function
        (err) {
          if (err) { return done(err) }
          write(client, new PubRec(packet), done)
        })
      break
    case 1:
      write(client, new PubAck(packet), function (err) {
        if (err) { return done(err) }
        client.broker.publish(packet, client, done)
      })
      break
    case 0:
      client.broker.publish(packet, client, done)
      break
    default:
      // nothing to do
  }
}

```

broker.publish 函式依 QoS 的不同，行為有所差異。主要都是透過 fastseries，依序進行 storeRetained(若有設定 retain，則存入 persistence 中)，emitPacket(向 mqemitter 真正發出訊息)，enqueueOffline(存入 persistence 中) 與 callPublished(產生 publish 事件供

系統管理使用) 等動作。上面的動作中，如果是 QoS0，就不會有 enqueueOffline，若為 QoS1-2 的話，就需要暫時存放訊息於 persistence 中，以備訊息丟失時，可以取得。接下來，如果先前在 mqemitter 中有註冊對應的 topic，此時 mqemitter 會呼叫相對應的訊息處理函式，依據 QoS 設定不同，可能為 deliverQoS 或 deliver0，透過這二個函式，結合 write.js 的功能，就能向 subscribers 進行派送匹配到的訊息。

到目前為止，我們已經將 Aedes 的原始碼架構與訊息流程做了完整的解析，接下來可以對程式碼的修改進行說明。首先面臨到的問題是，目前 Aedes 只支援 MQTT 3.1，所執它採用的 mqtt-packet 專案，可以透過改動 opt 的方式，將封包格式轉換為 5.0，所以我們在 Client 類別的建構函式中加以調整：

```

// ...in client.js
const opts = { protocolVersion: 5 };
this._parser = mqtt.parser(opts);
// ... other code

```

此外，在送出的部份，要送出 MQTT 5 的封包格式，write.js 中 mqtt.writeToStream 需要修改，不然接收者會收到錯誤的封包。

```

// in write.js
const result = mqtt.writeToStream(packet, client.conn, {protocolVersion: 5})

```

本計畫所要完成的重點主要有二個：回執機制與斷線重連的補送機制。其中，回執機制需要在 writeQoS 中進行修改與回執發送，因為在這裡才會知道確切的發送結果，也才能發送正確的回執。發送回執時，只要使用 broker.publish 函式，將訊息發佈到指定的 topic 即可，預設為 \$SendToEndResponses/\$P/\$t。而 retainMode 的處理，則涉及了 deliverQoS 和 memory persistence 的增改，我們將比較重要的部份列在下面的程式片段。

```

// in client.js, function deliverQoS
if (toForward) {
  ...
  packet.writeCallback = cb
  // 清理CleanSession 0為保留 1為要清理乾淨 不用儲存任何東西
  // packet.retain 1為retain 重新連接後要重送的
  if (that.clean || packet.retain) {
    writeQoS(null, that, packet)
  } else {
    //準備發送publish需要給封包加上messageID
    broker.persistence.updateSubOutbox(that, packet, writeQoS)
    ...
  }
}
}

```

Distributed Systems

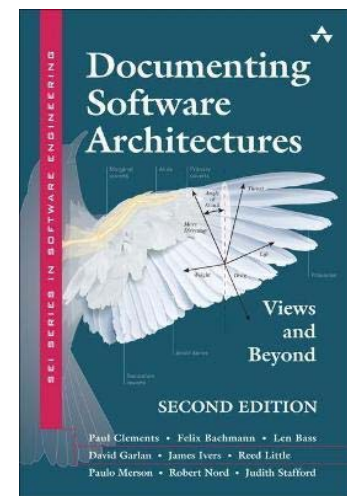
Describing Software Systems

Chun-Feng Liao

廖峻鋒

Dept. of Computer Science
National Chengchi University

設計軟體但沒有妥善描述它，就像是站在暗處對路人拋媚眼 – 你知道自己在做什麼，但其它人都不知道你在做什麼
(you know what you are doing, but nobody else does)



「表述」軟體系統的重要性

設計軟體但沒有妥善描述它，就像是站在暗處對路人拋媚眼 – 你知道自己在做什麼，但其它人都不知道你在做什麼
(you know what you are doing, but nobody else does)

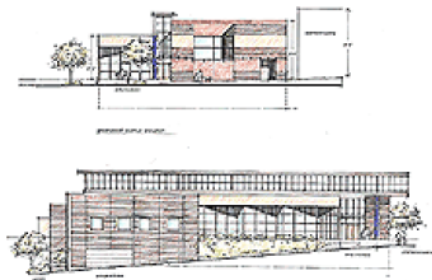
- 欠缺對軟體系統的表述能力，那麼即使這個軟體設計再精妙，也無法為人所理解認同，自然軟體就無法展現價值
- 許多軟體工程師包怨企業，「只喜歡硬體，不懂軟體的價值」
 - 是否欠缺如何清楚展現軟體設計的能力？
- 表述能力對反向工程也很重要
 - 理解系統後，妥善地以易懂方式表達出來，才有機會將知識分享給團隊

描述軟體系統的目的

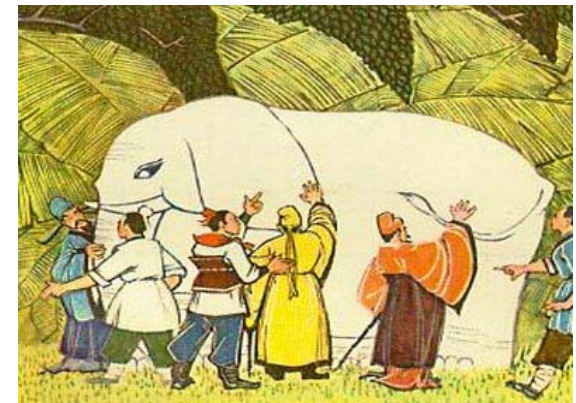
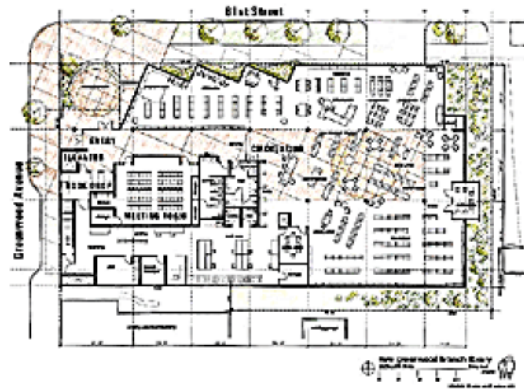
- 學習理解
 - 反向工程 / 程式碼解析
- 設計工具 (design by writing)
 - 有助更嚴謹思考系統設計
 - 軟體性質分析 (*-ility)
- 溝通橋樑
 - 學術論文中對軟體設計的表述
 - 職場中分享設計技巧或與其它架構設計人員相互學習
 - 訓練新進人員

系統設計文件

- View
 - 用特定「觀點」或用特定「描述風格」來描述此系統
 - 由此「觀點」、「描述風格」可看出軟體系統的性質
 - 包含圖與文字描述



(Buffalo Design © 2002. Images used with permission.)



如何描述一個軟體系統的架構

- 選擇“觀點 (Views)”

- 此views應對整體軟體系統的Quality Attributes有影響

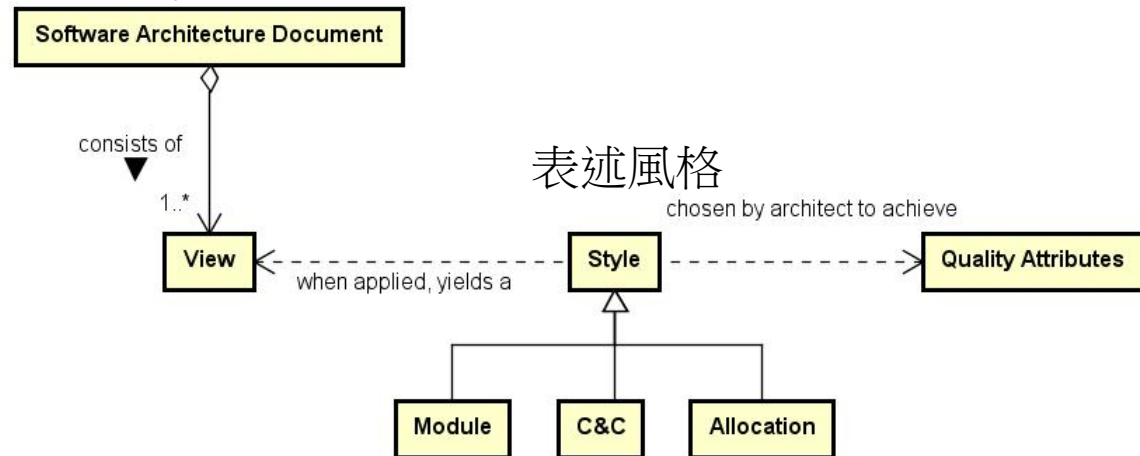
- Ex: 多快、多彈性、多穩定、多安全

- 畫圖並以文字論述設計

- 動靜態觀點

- 評估與討論

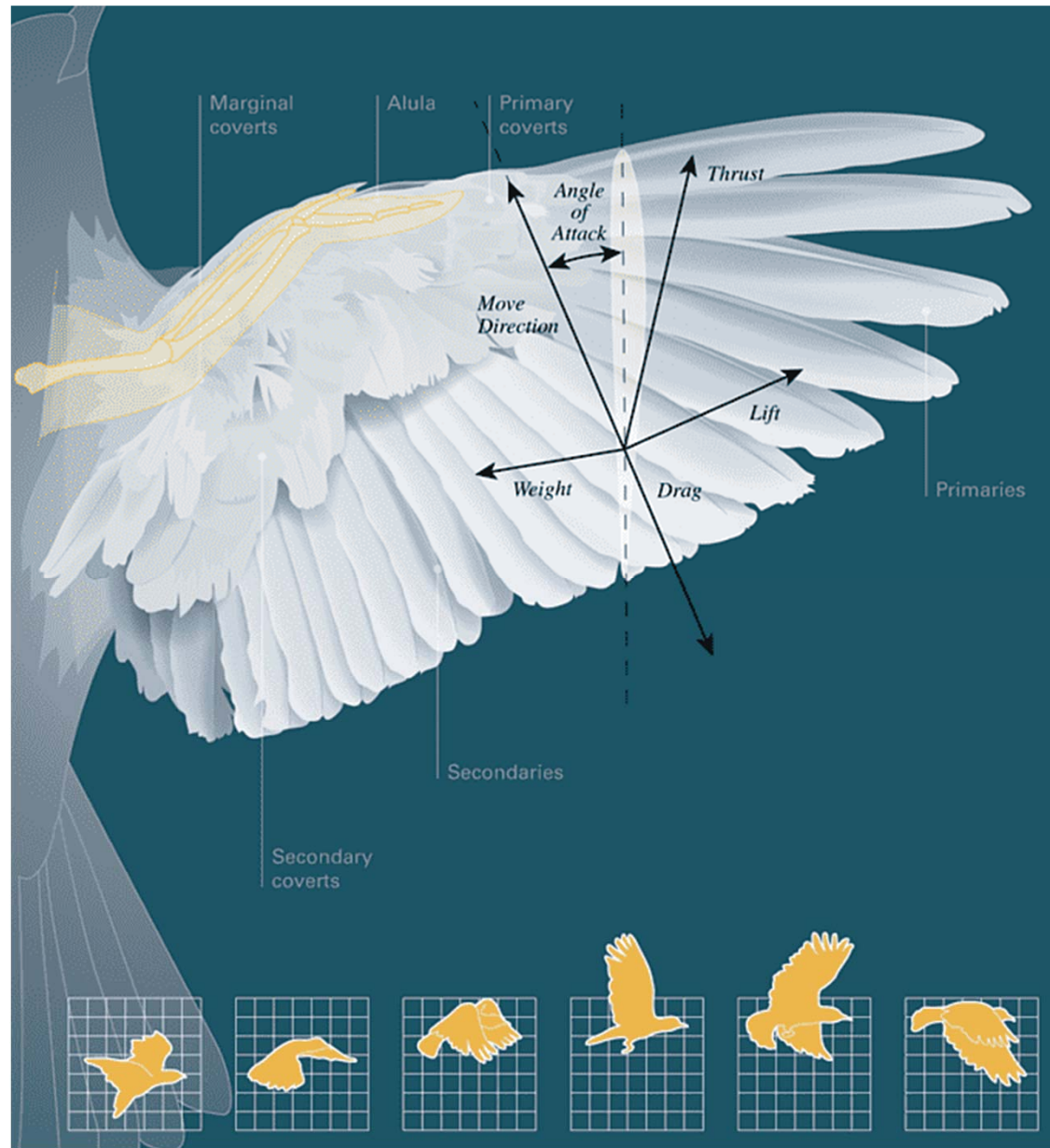
靜 structure; source code
動 behavior



Ingredients of Views

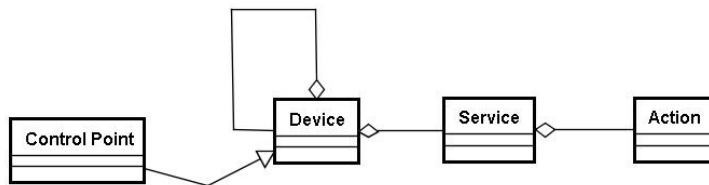
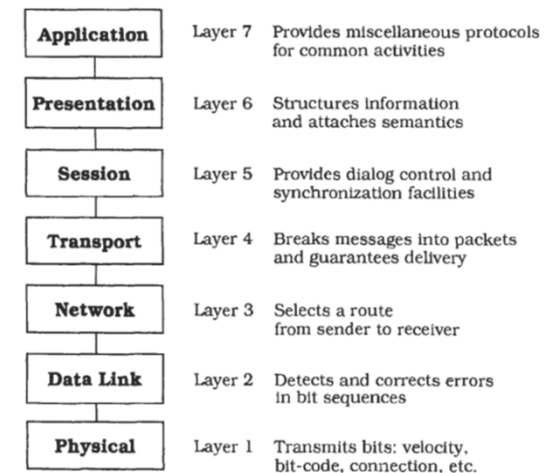
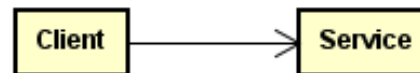
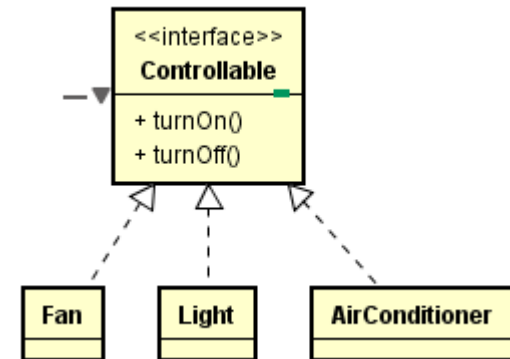
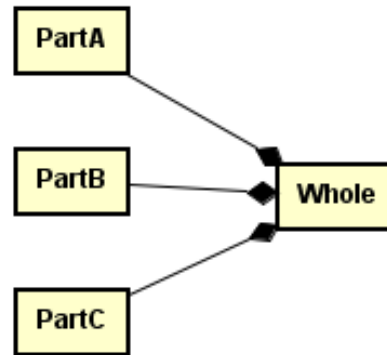
- Diagrams 圖
 - Graphical presentations (不一定要是 UML)
- Descriptions 文
 - **Explains** and **defines** the elements shown in the view and lists their properties
 - Possible **extensions**: explain any built-in mechanisms for tailoring the architecture
 - **Design rationale**
 - Explain the choice/trade-offs of the approach
 - How quality attributes are satisfied using the approach

unified
modeling
language



描述的結構風格 (Structural Styles)

- Decomposition
- Use (依賴)
- Hierarchy
- Layer
- Domain model

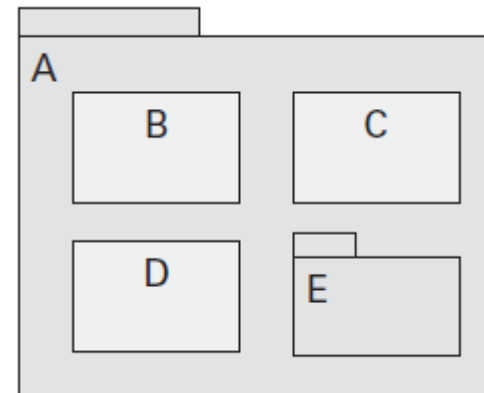


Astah UML

<https://astah.net/pricing/academic/>

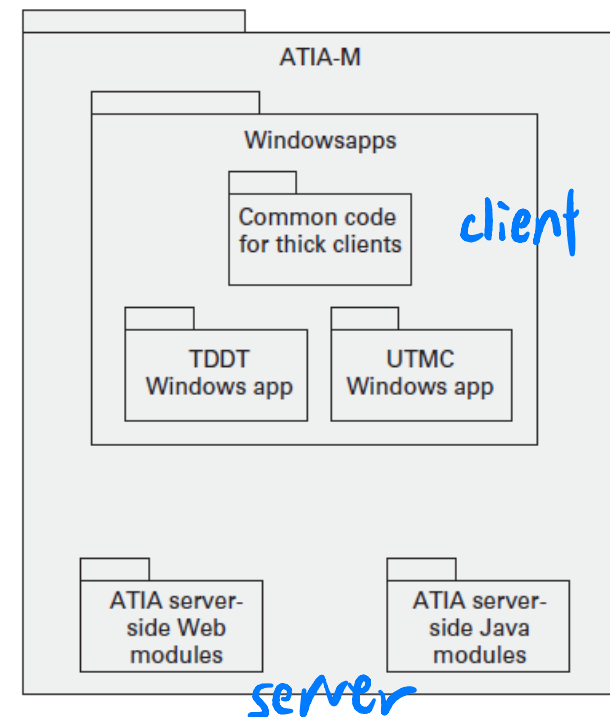
Decomposition

- 意義
 - 由源碼模組觀點提供彼此的包含關係
- 功能
 - 提供軟體源碼概觀
 - 工作分配



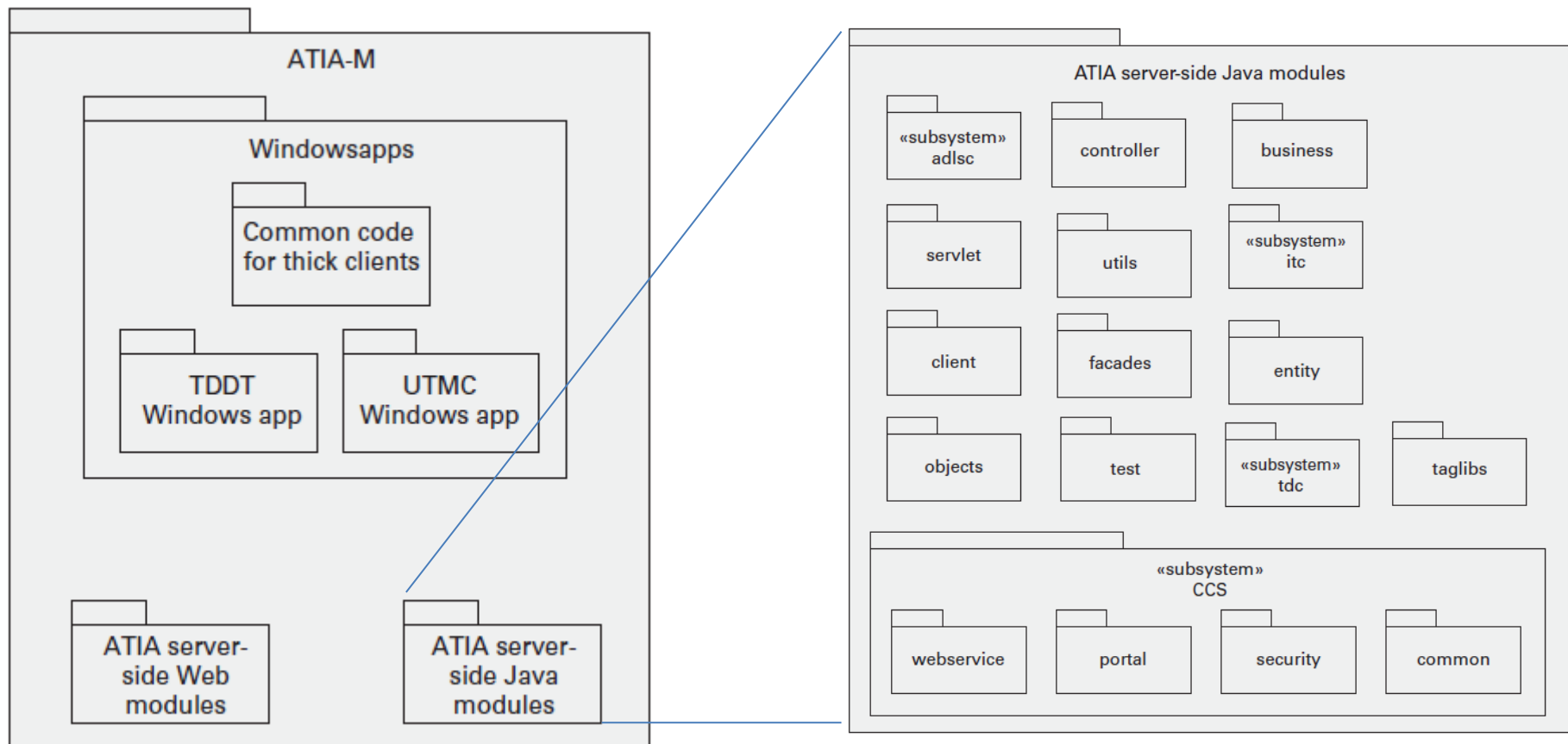
Case: ATIA-M

- Army Training Information Architecture-Migrated (ATIA-M)
- A web application that supports training in the U.S. Army
- Structure
 - Client: C#
 - Server: Java EE/ Web Services



Case: ATIA-M

模組可以再展開，提供更詳細資訊
如此一來，可以在不同的精細度層級來畫圖

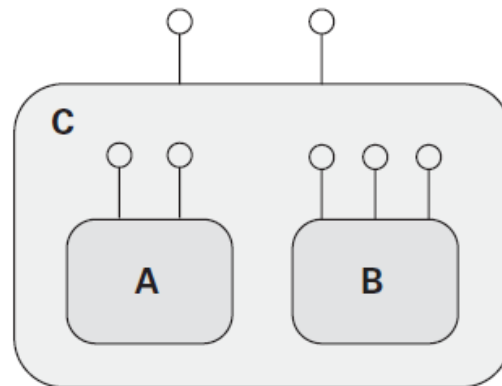


Decomposition

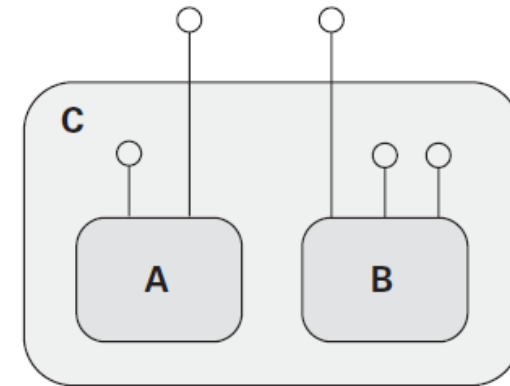
- Modules
 - Unit of **source code**
 - Ex: classes, packages/namespaces
 - Each module has a set of responsibilities

Module C provides its own interface, hiding the interfaces of modules A and B.

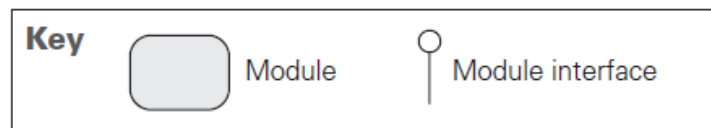
encapsulation



(a)



(b) Module C exposes a subset of the interfaces of modules A and B as its interface



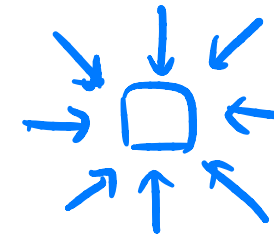
Uses

- 意義
 - A form of depends on relation
 - Shows how modules depend on each other

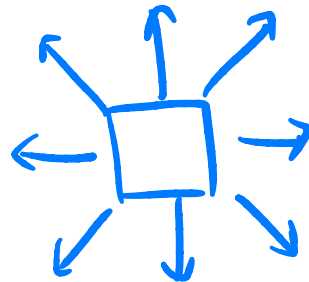
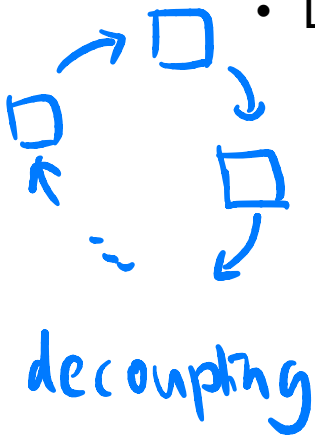
- 功能

- 除錯與測試
- 源碼新增/變動分析
- 看出不佳的設計

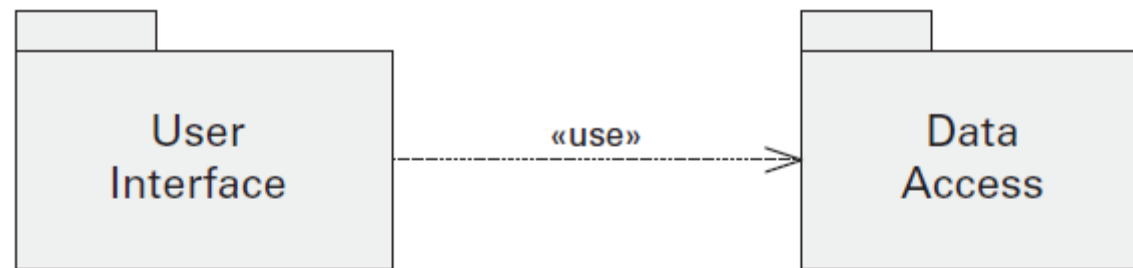
(x) broad fan-in



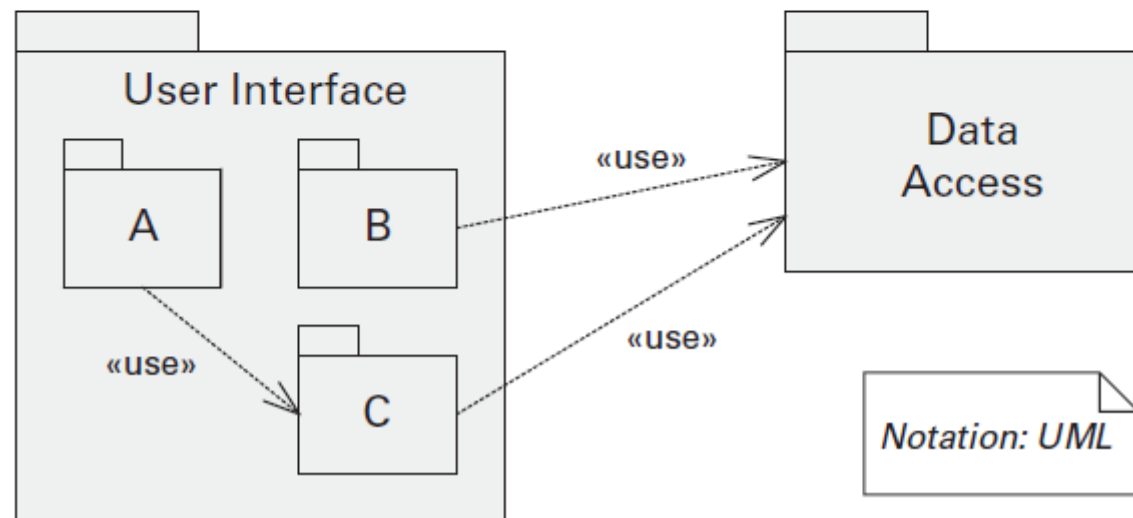
- Loop, broad fan-out, long dependency chains



“Uses” with UML



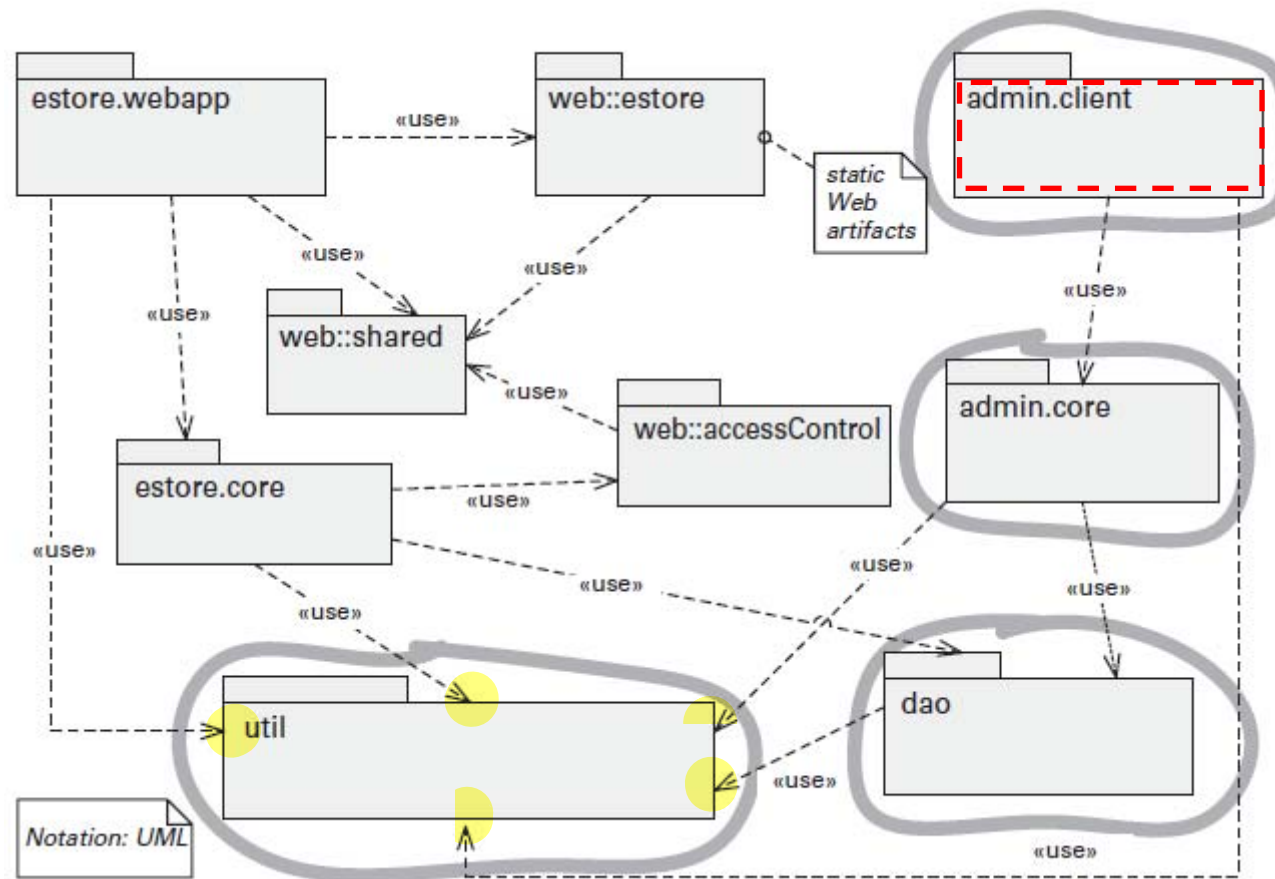
Notation: UML



Notation: UML

Dependency Chain

要執行admin.client，所需要用到的模組

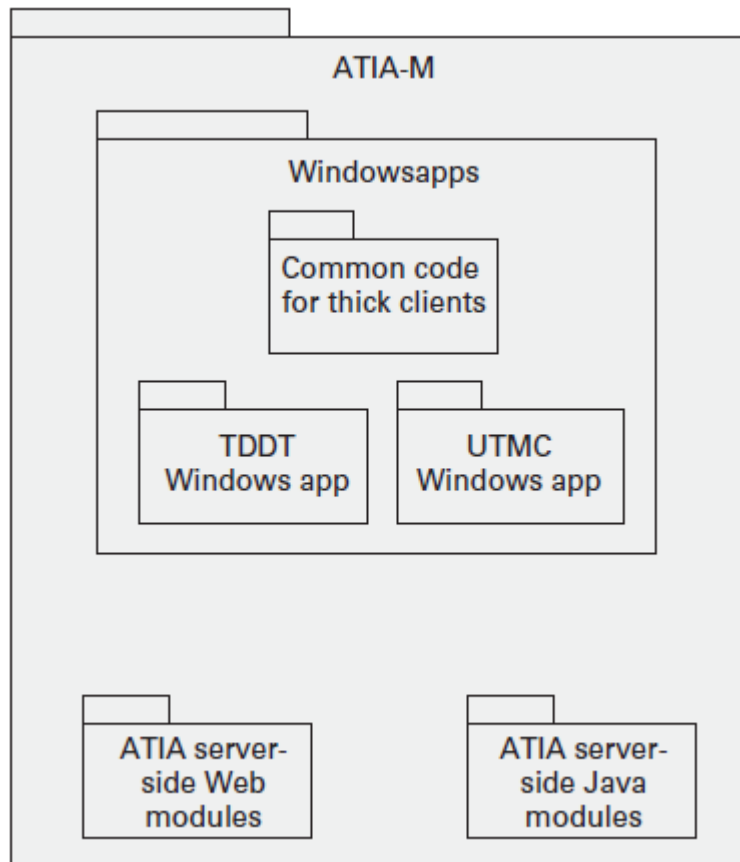


broad fan-in

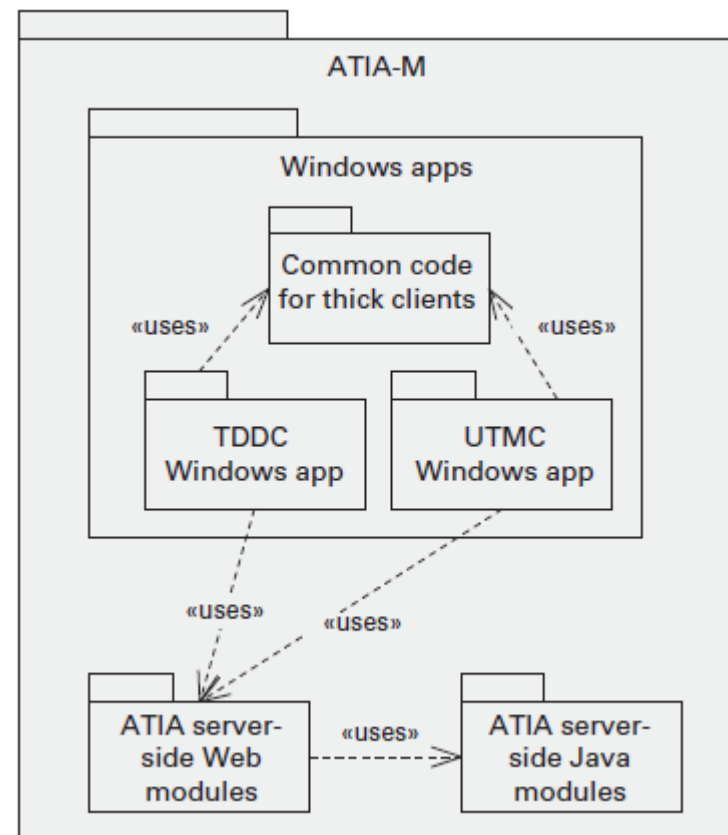
SOLID Principles

Case

decomposition

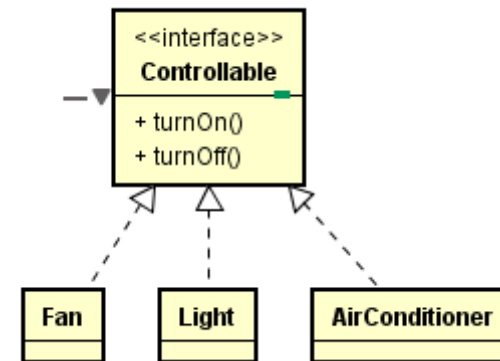


decomposition + uses



Generalization

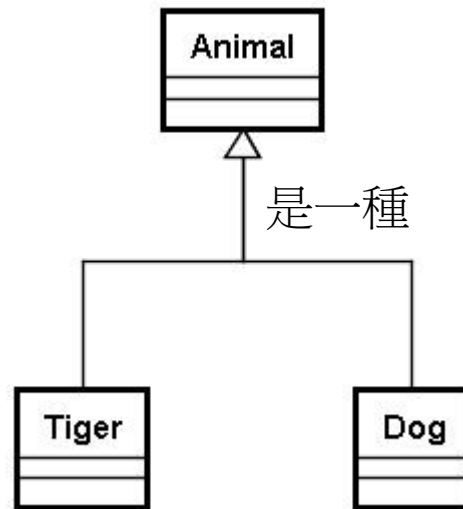
- 意義
 - Is-a
- 功能
 - 表述物件導向設計
 - 表達擴充方式(extension)
 - 列舉可抽換的組件



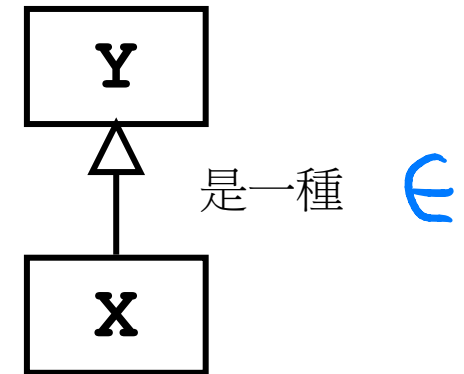
“X是一種Y”

- “X是一種Y”
 - 比較好的說法: X可以被當做Y來用

想一想: 為何X可以被當做Y來用?



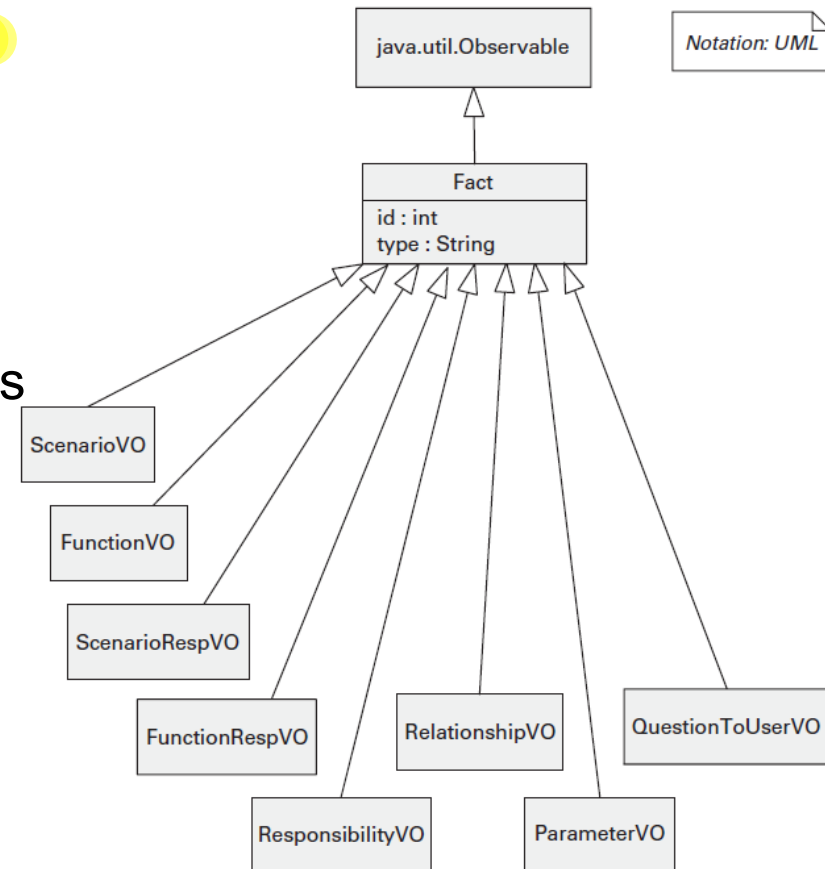
Tiger是一種動物
Dog是一種動物
動物都會走
動物都會吃



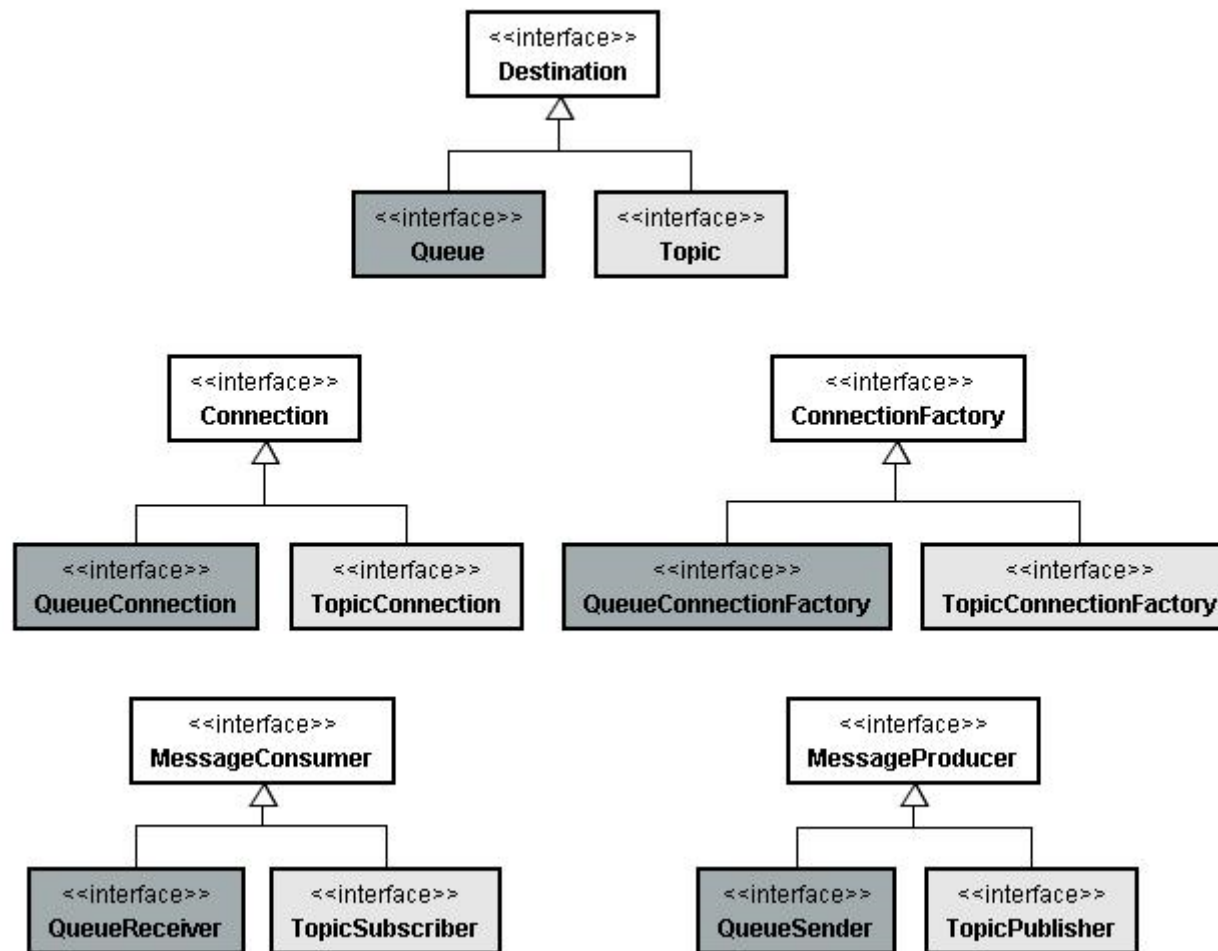
Case: ArchE

- SEI Architecture Expert (ArchE) tool

- Used to create the architecture design for a system
- Uses a rule engine that manipulates “facts”
- Specific functionality to deal with different kinds of facts

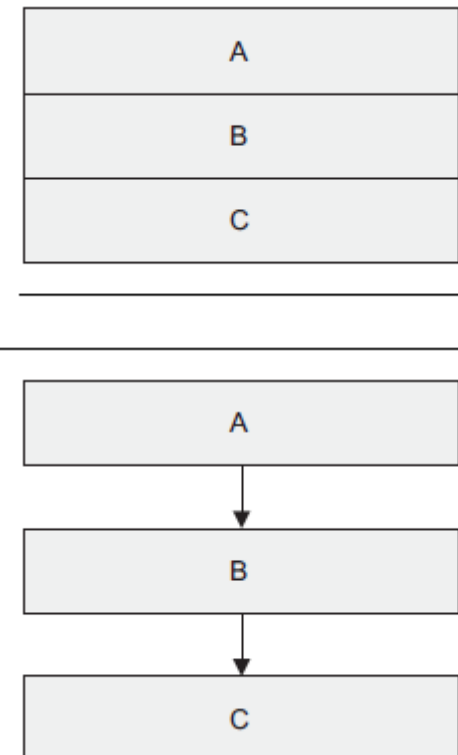


Case: JMS Java Message Service



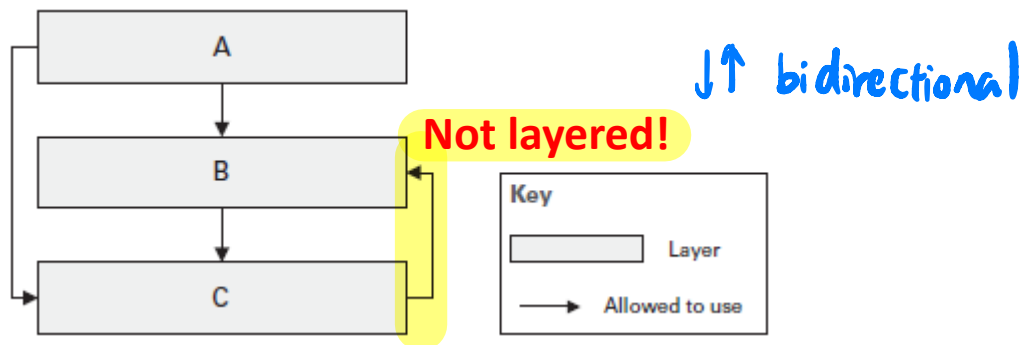
Layer

- 意義
 - One of the most commonly used views
 - Each layer denotes a group of cohesive modules
 - Layers reflect “strict ordering relation”
- 功能
 - Promote modifiability and portability
 - Achieving separation of concerns



Layer

- Layering
 - Inside a layer: modules offer a cohesive set of services
 - Among layers: **unidirectional** *allowed-to-use* relation
- Constraints
 - One module is allocated to exactly one layer
 - At least two layers (typically three or more)
 - The allowed-to-use relations should not be circular



Benefits of Layers

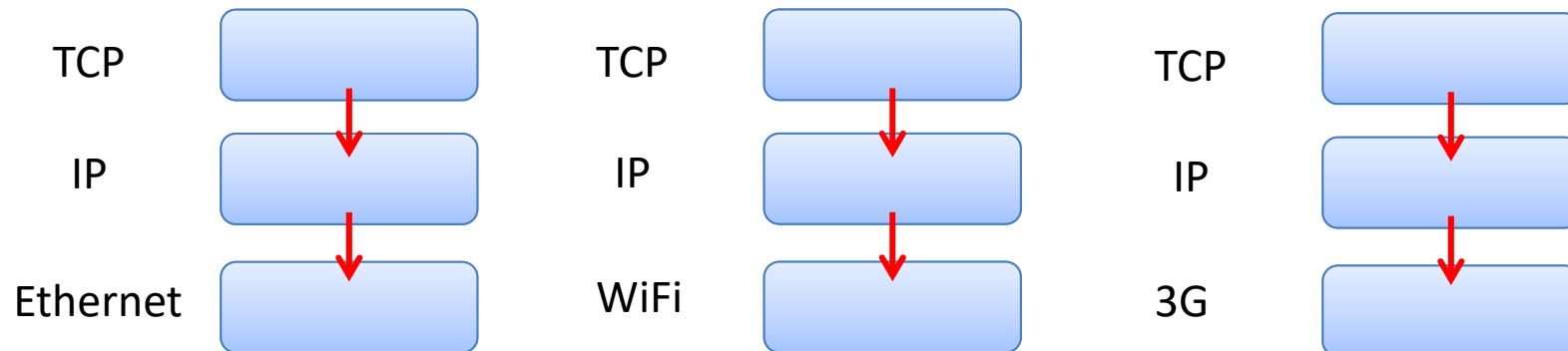
- Support for standardization
 - Clearly-defined and commonly-accepted levels of abstraction enable the development of standardized tasks and interfaces
 - Ex: OSI and POSIX
- Dependencies are kept local
 - Portability up
 - Testability up

Open
Systems
Interconnection
Model

Portable
Operating
System
Interface
X

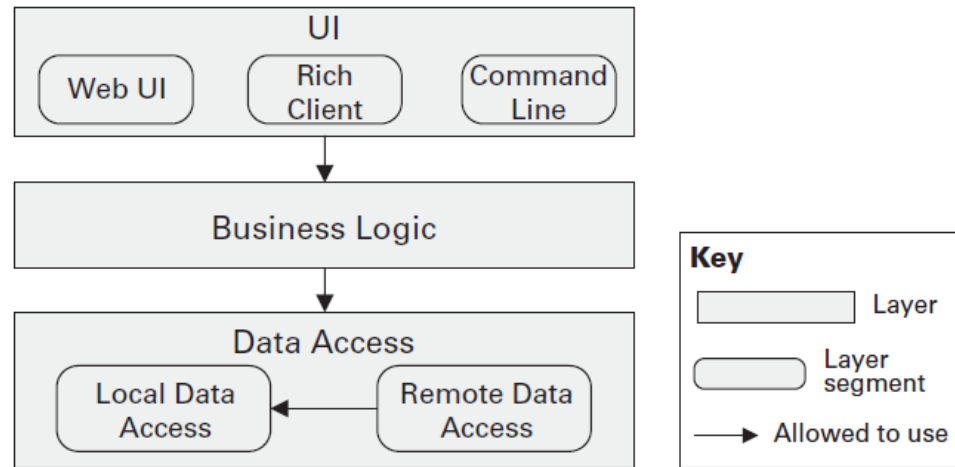
Benefits of Layers

- Exchangeability
 - Individual layer implementations can be replaced by semantically-equivalent implementations without too great an effort.

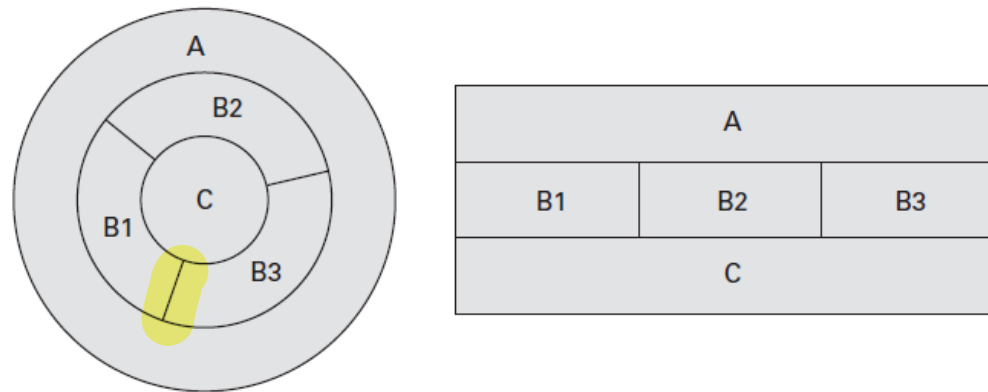


表達Layer的方式

- Segmented



- Rings



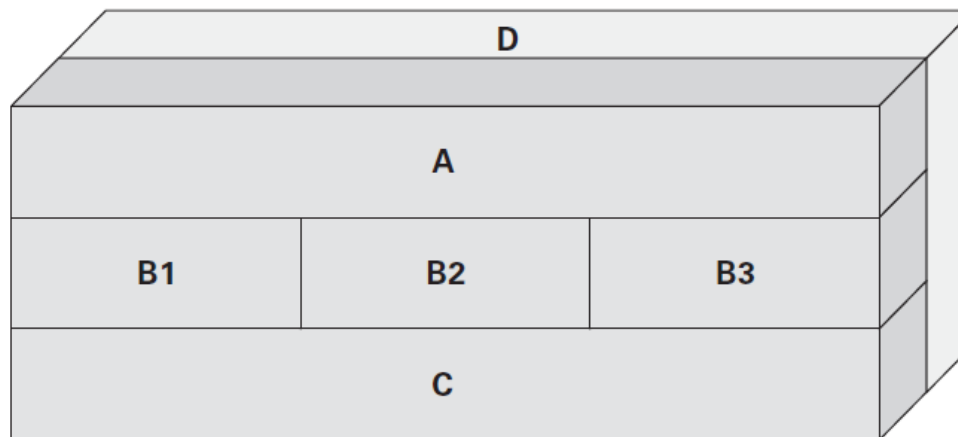
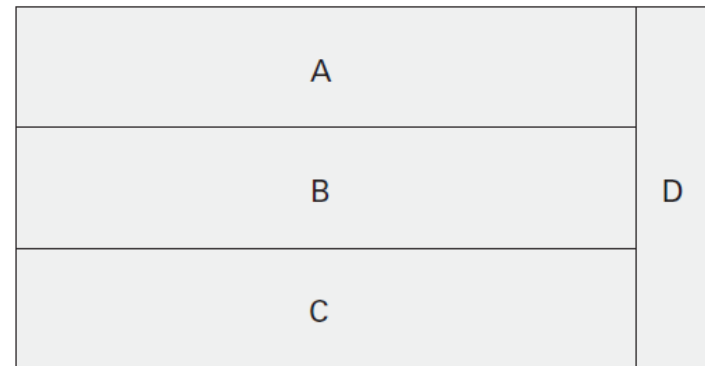
表達Layer的方式

- Layer with a sidecar

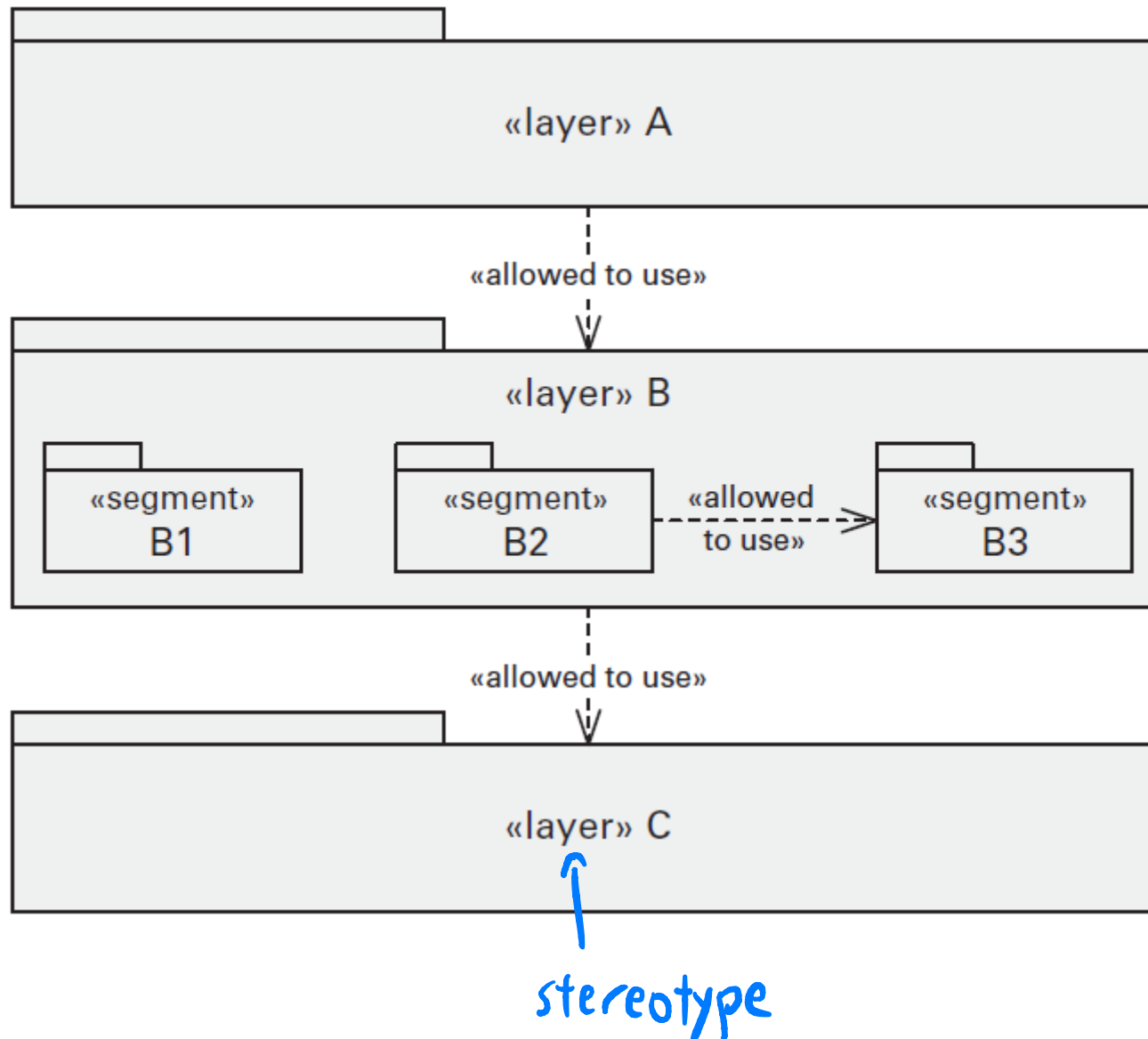
D is used by all other layers

Or

D uses all other layers

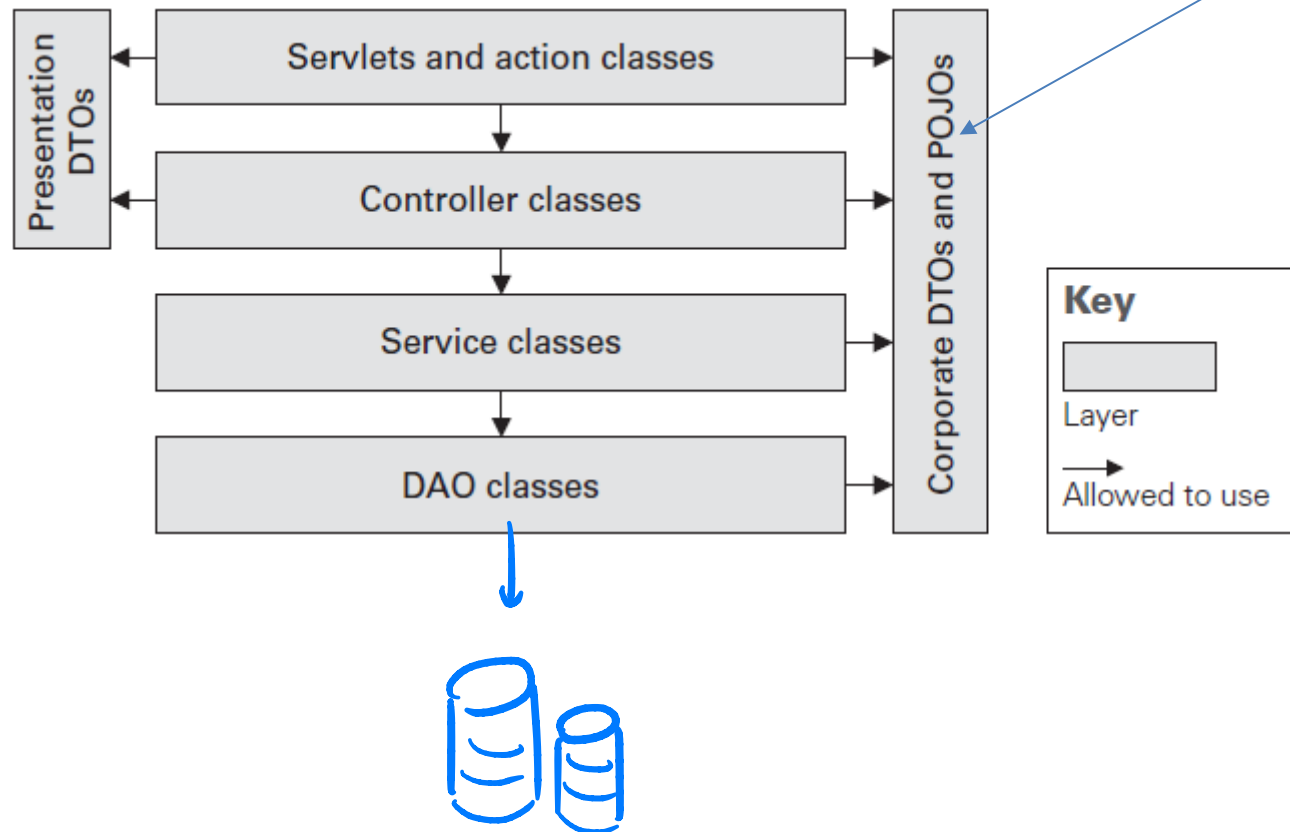


表達Layer的方式 (UML)

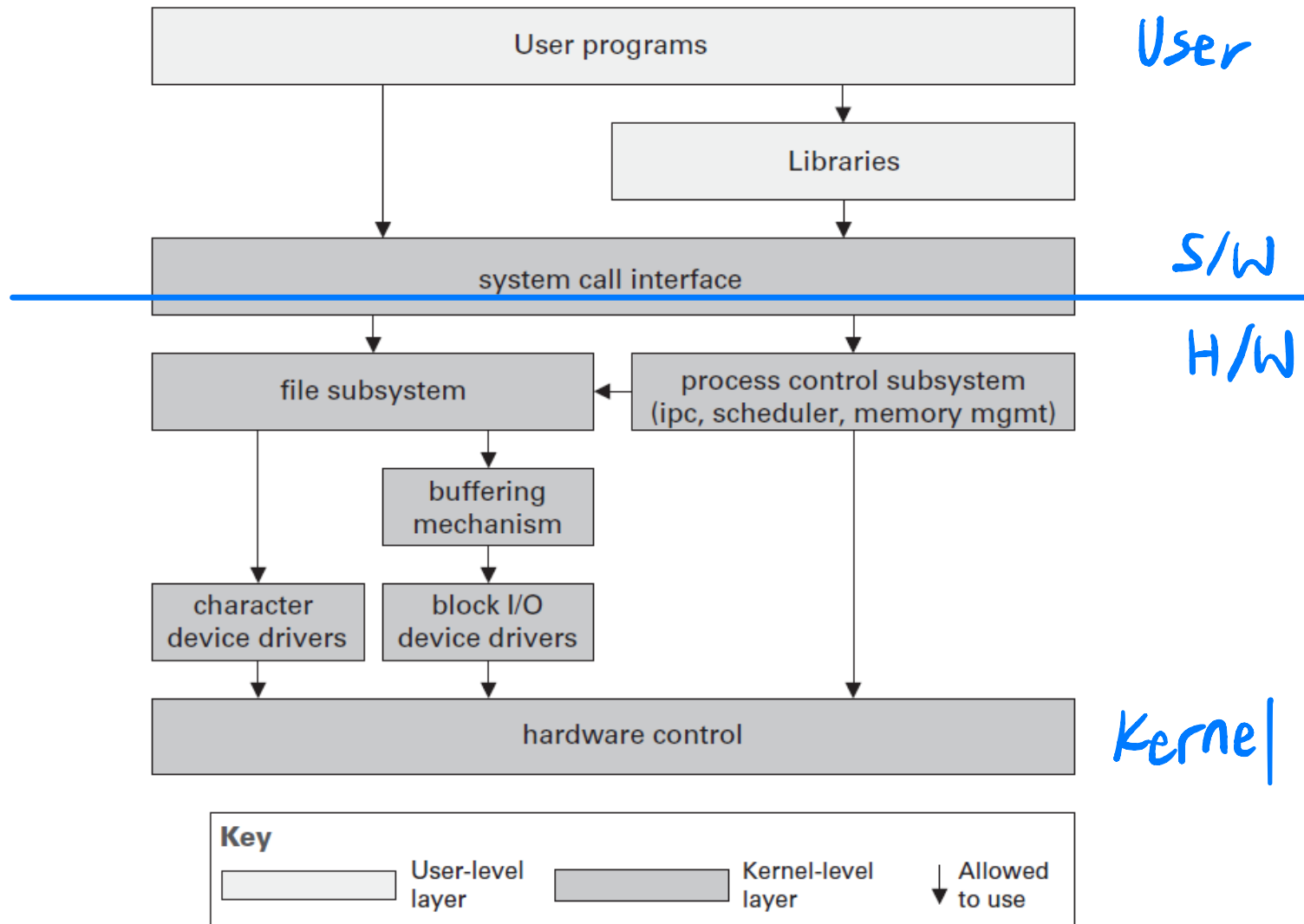


Enterprise Edition Case: Java EE

Expressed using domain model

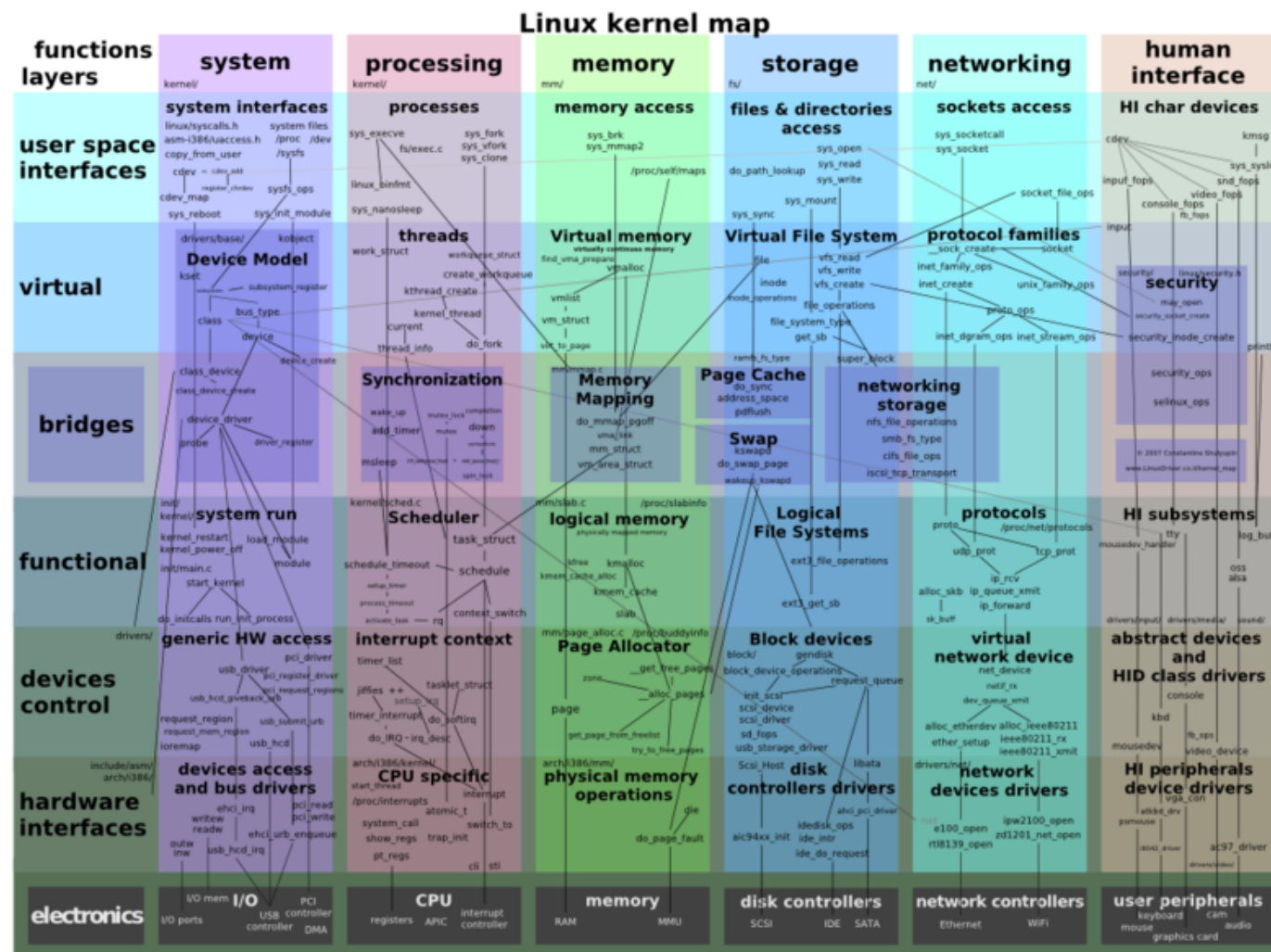


Case: Unix System V



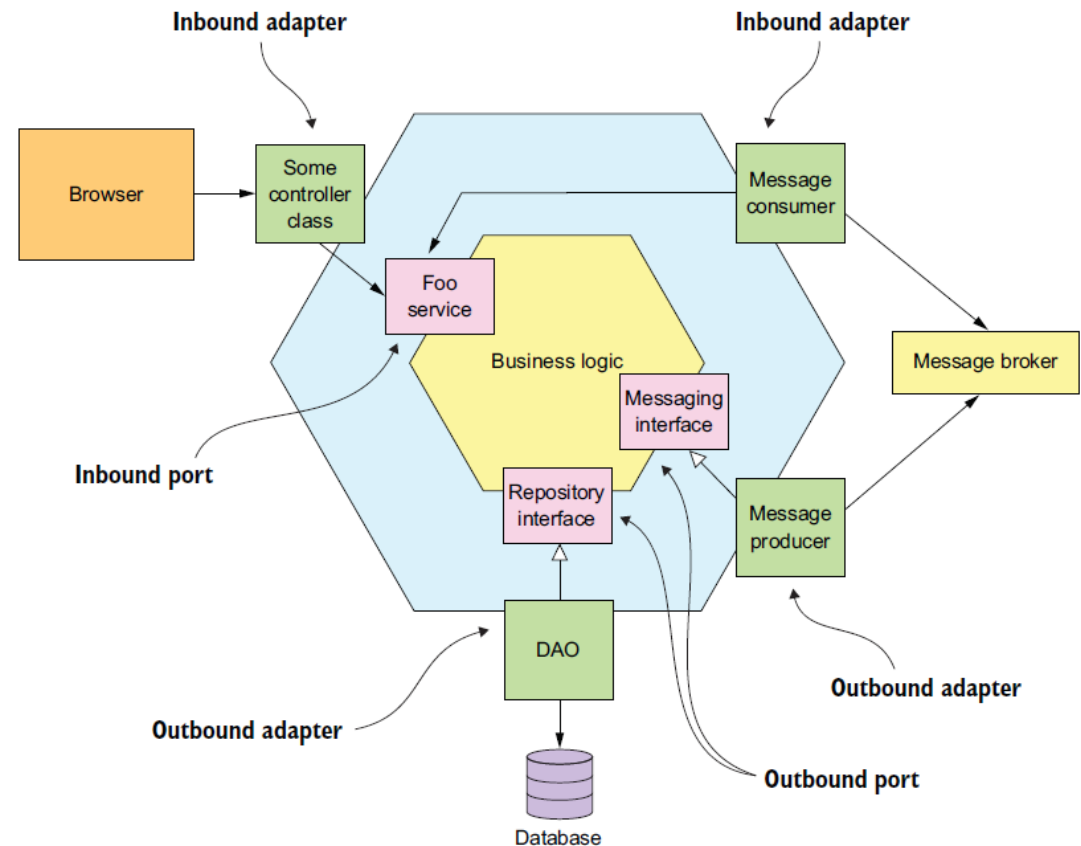
Case

- Linux kernel architecture (a relaxed layer pattern)



Hexagonal Architecture Style

- An alternative to the layered architectural style
- Core components
 - Core
 - Business logics
 - Ports (pink)
 - Access or be accessed by external services
 - Adapters (green)
 - In-bound adapters
 - Out-bound adapters



Domain Model

- 意義
 - 又稱data model
 - 以名詞和名詞間關係(含數量關係)描述業務邏輯
 - 多以ER Diagram或UML類別圖實現
- 用途
 - 特定領域知識的粹取與保存
 - 在傳統應用程式中常據以建立資料表(Tables)
 - 開發過程與stakeholder共同的語言
 - 也可用來表達系統設計的核心概念
 - model引導實作程式碼，而實作過程也會引導出更好的model

スパイラル



社長
瀬名洋介
(尾上松也)



役員
加納一成
(井上芳雄)

電腦雑伎集団



社長
平山一正
(土田英生)



平山の妻
平山美幸
(南野陽子)

太洋証券



営業部長
広重多加夫
(山崎銀之丞)

フォックス



社長
郷田行成
(戸次重幸)



小料理屋 女将
智美
(井川 遼)

買収

アドバイザー契約

東京セントラル証券

プロパー社員



営業企画部調査役
森山雅弘
(賀来賢人)



営業企画部員
浜村 瞳
(今田美桜)



営業企画部長
半沢直樹
(堺 雅人)

夫婦



半沢の妻
半沢 花
(上戸 彩)

敵視

同期

東京中央銀行



証券営業部長
伊佐山泰二
(市川猿之助)



副頭取
三笠洋一郎
(古田新太)



半沢の同期
渡真利 忍
(及川光博)



頭取
中野渡 謙
(北大路欣也)



取締役
大和田 暁
(香川照之)

敵対

忠誠？

銀行出向組



営業企画部次長
諸田祥一
(池田成志)

出向組とプロパー社員との確執

日曜劇場

半沢直樹

帝国航空再生タスクフォース



リーダー
乃原正太
(筒井道隆)

政府



国土交通大臣
白井亜希子
(江口のりこ)



進政党 重鎮議員
箕部啓治
(柄本 明)

金融庁



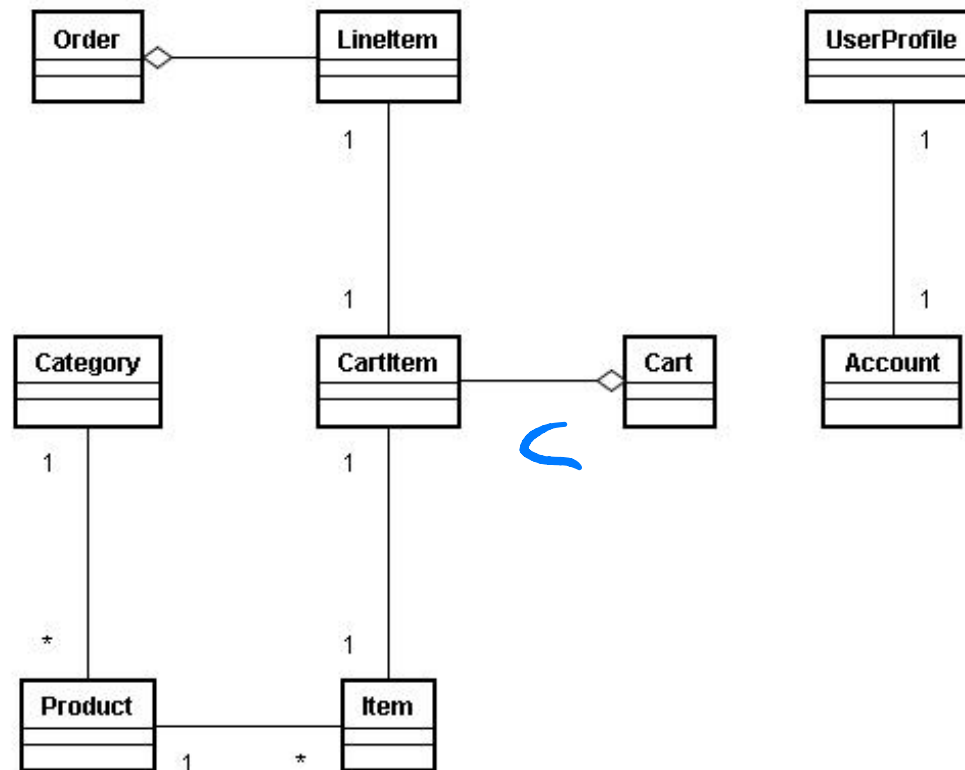
検査局 主任検査官
黒崎駿一
(片岡愛之助)

人物相関図

Domain Model與應用程式的關係

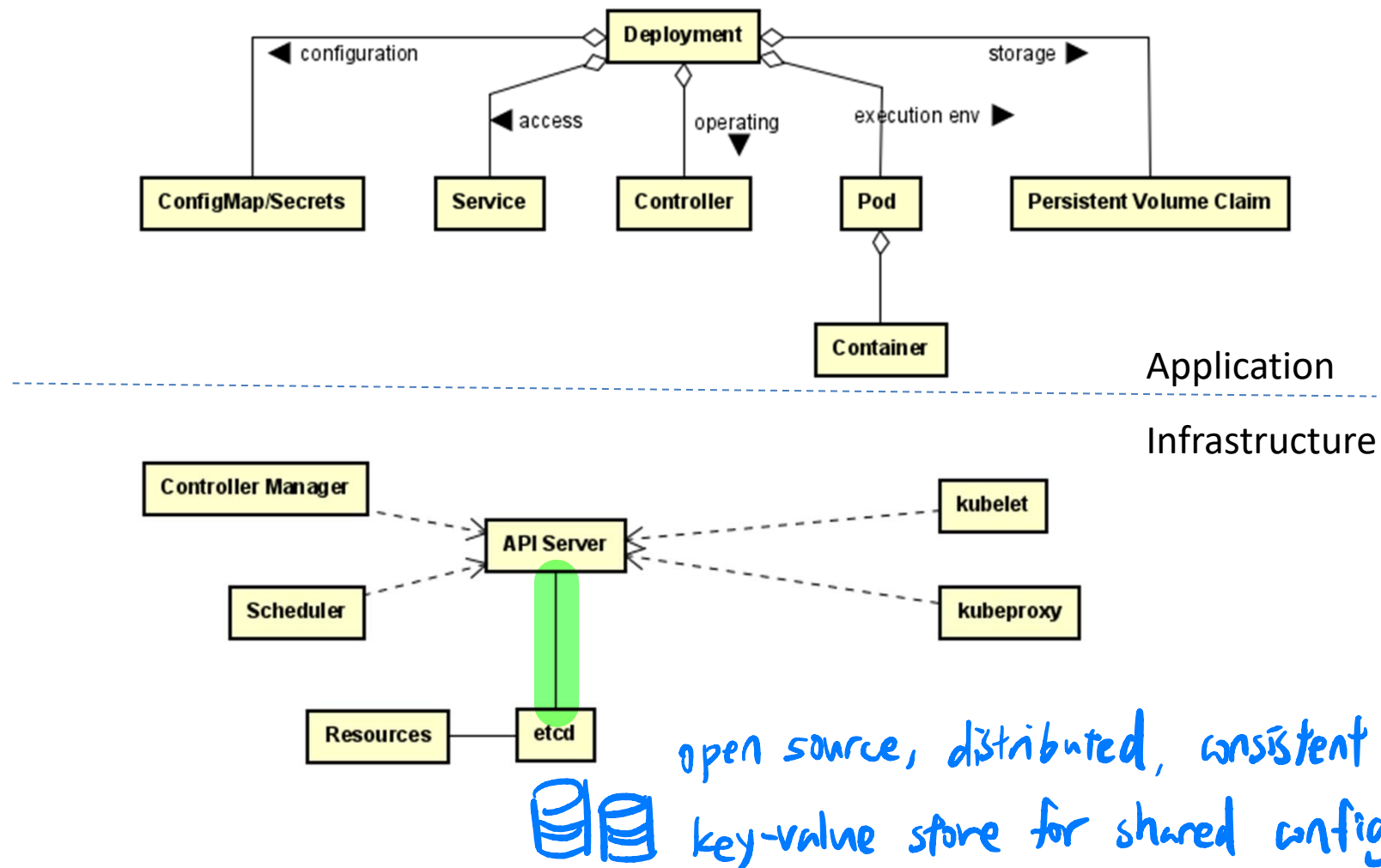
- UI and Domain Model
 - UI presents concepts of a Domain Model
 - Allows user to perform various actions on the model
- Application Services and Domain Model
 - Called by UI
 - Coordinates use case tasks on the model

Domain Model Example: JPetStore

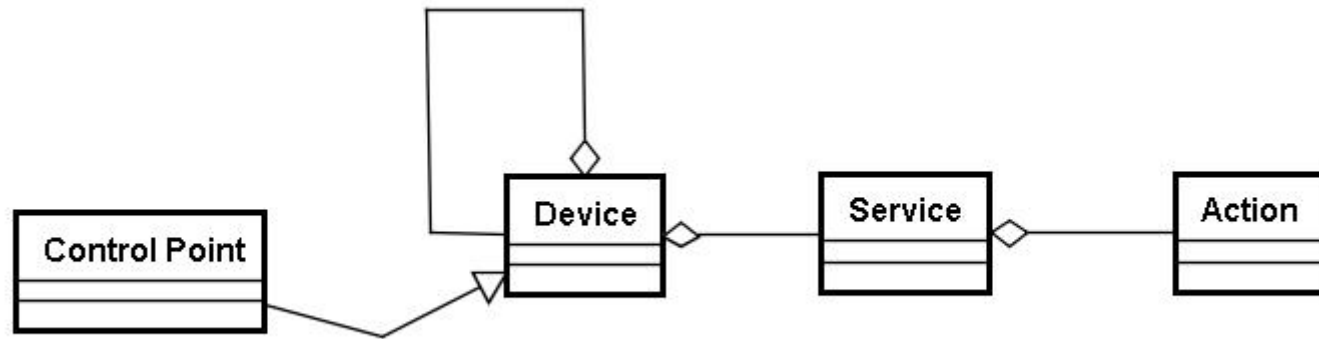


Domain Model Example: Kubernetes

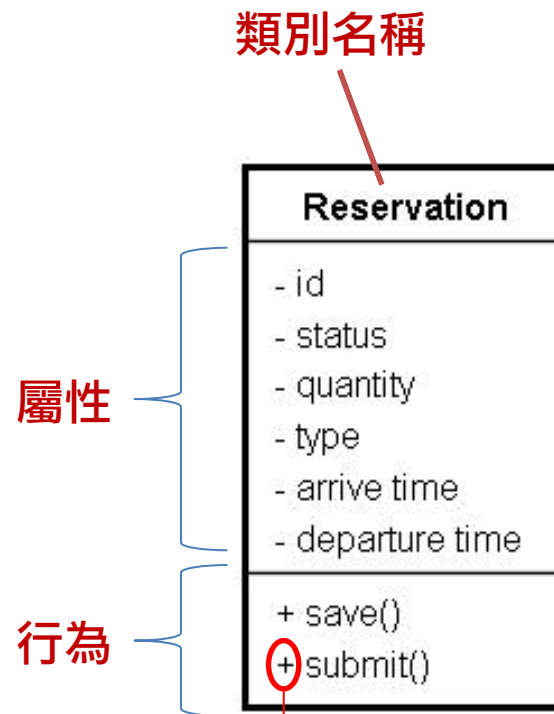
- Key concepts



Domain Model Example: UPnP



使用UML類別圖表述Domain Model

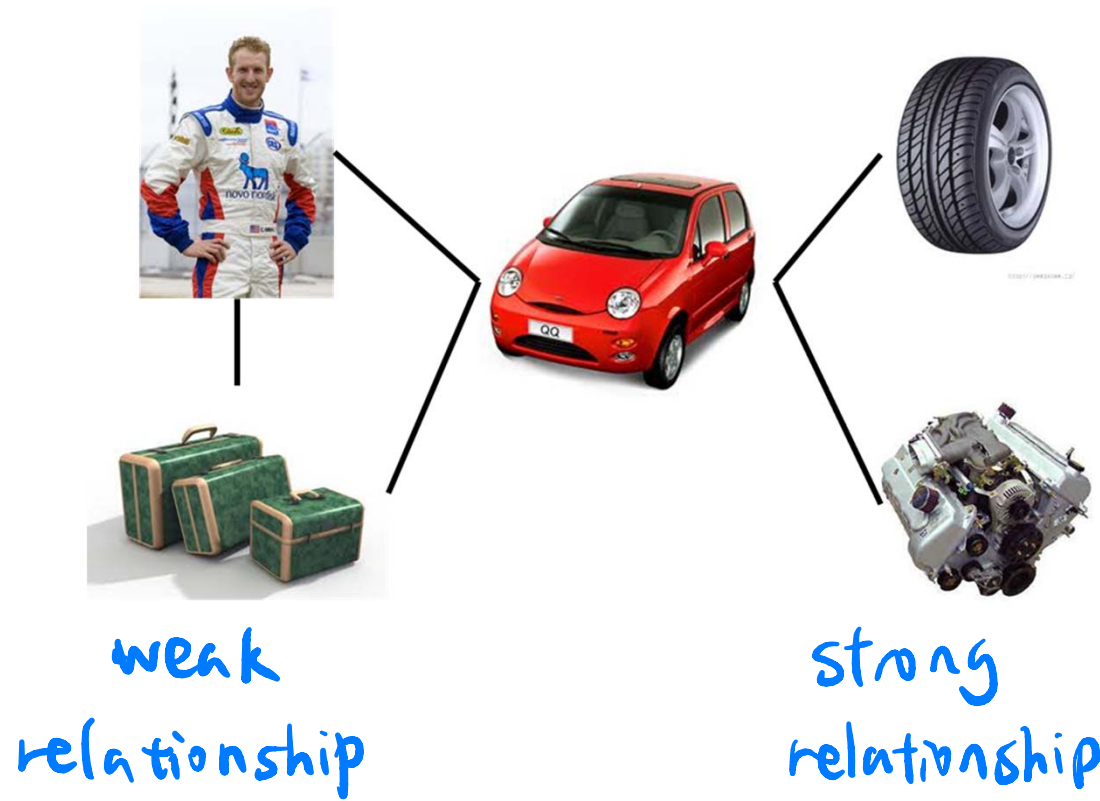


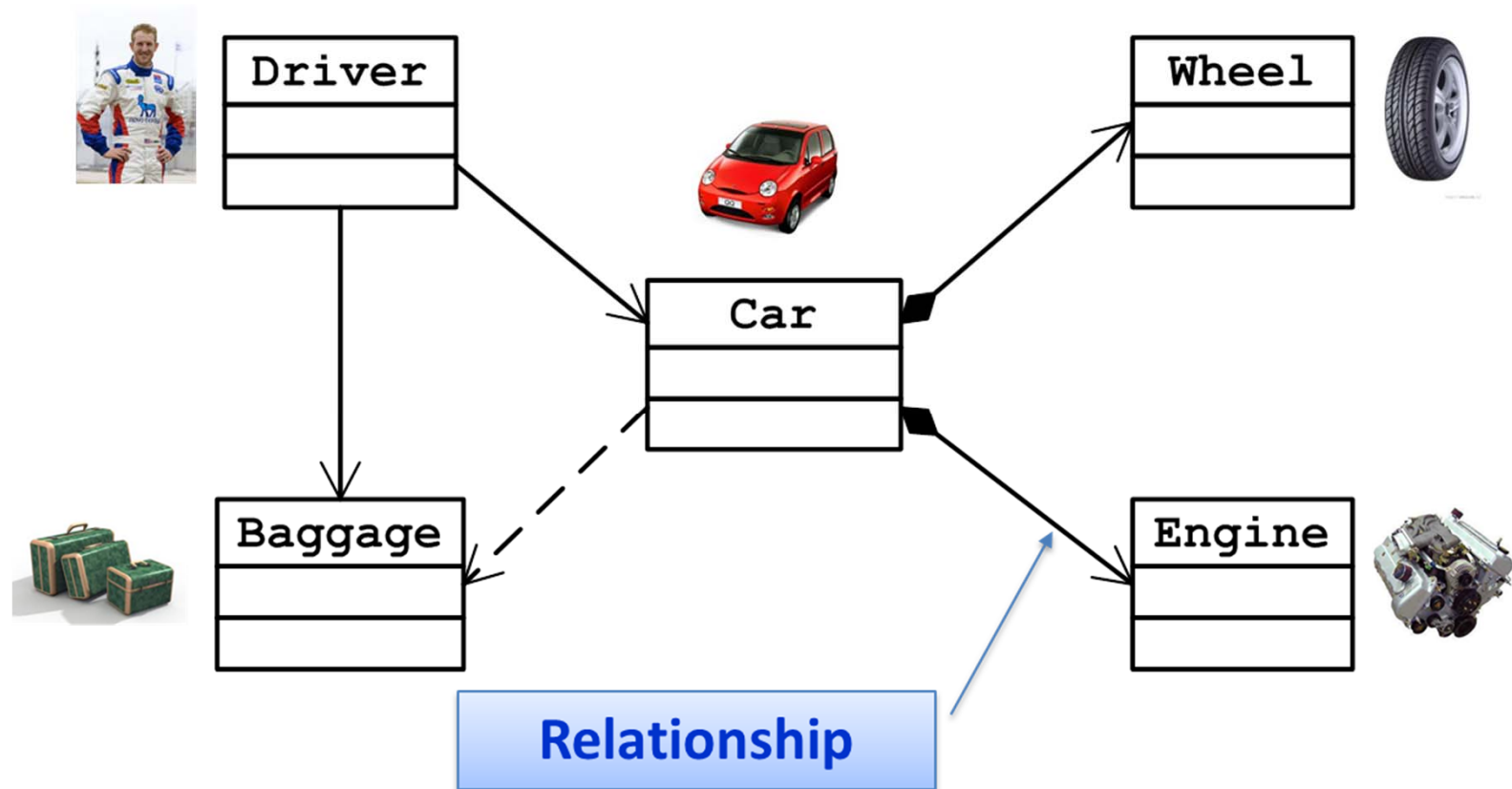
- + 代表public: 大家都可以存取
- 代表private: 只有在此物件內可存取
- # 代表protected: 只有此物件內及其子類別內可存取

Java

使用UML表達物件間的關係 (relationship)

- 系統的靜態結構
 - 一群彼此互動的物件
 - 互動的物件形成了許多不同的關係





使用UML表達relationship

- Dependency



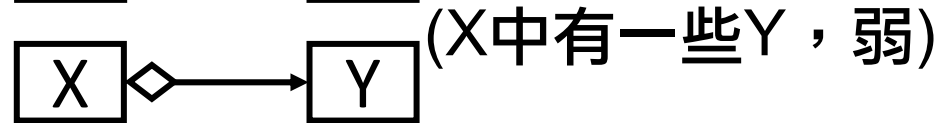
- Association



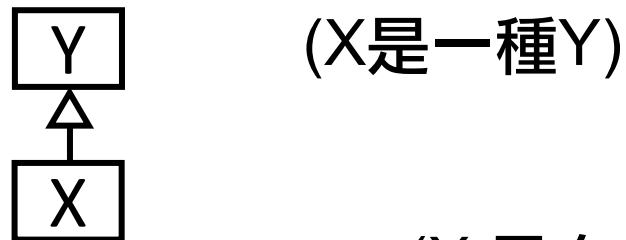
- Composition



- Aggregation



- Inheritance



- Implementation



Has-a關係

- Has-a關係
 - “X 中有一個 Y” 或 “X 中有一些 Y”
 - 又稱Composition或Aggregation



Has some

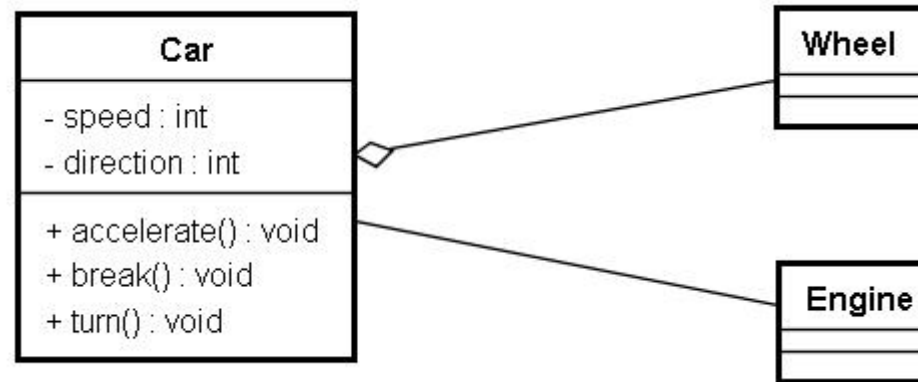


wheels

Has an



engine



Has some



wheels

Has an

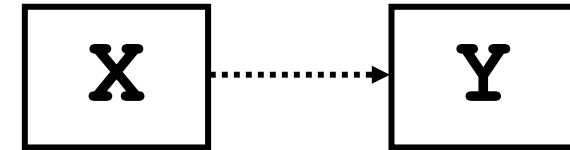


engine

“X使用Y” $\Leftrightarrow$ “X知道Y”

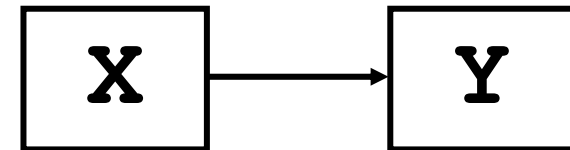
- X使用Y “Uses”

- Dependency
- X 類別呼叫了Y類別中的一個方法
- 短期關係

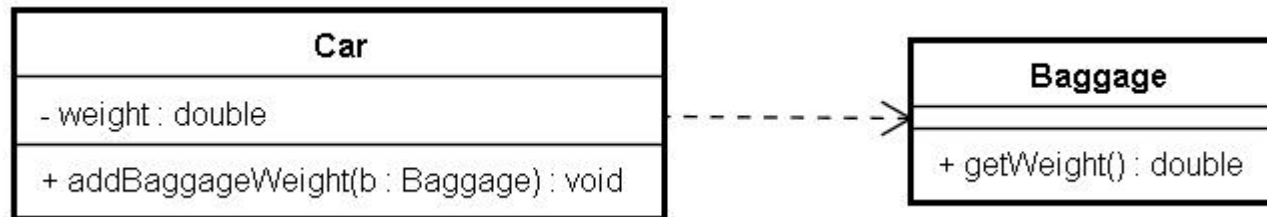


- X知道Y “Knows”

- Association
- X中會具有一個屬性，是指向Y的參考
- 此一參考使用X可以隨時呼叫Y中的方法
- 長期關係

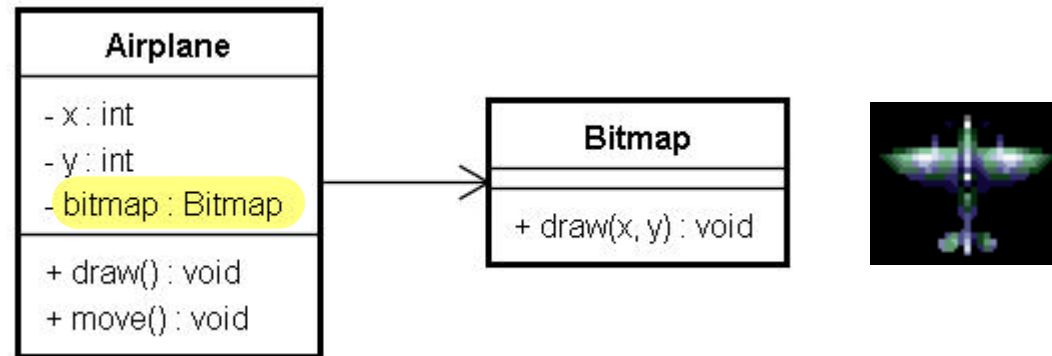


Demo: uses



"Car" uses "Baggage"
⇒ dependency (短期關係)

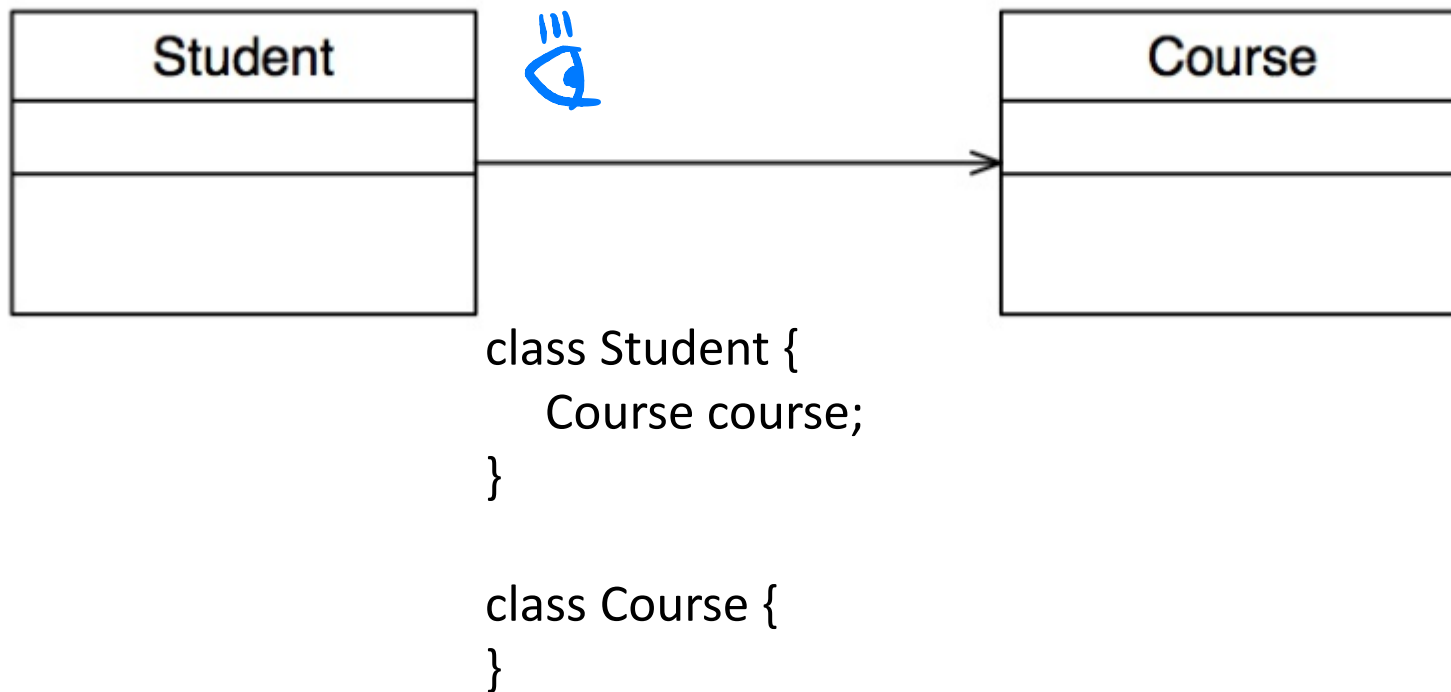
Demo: knows



"Airplane" knows "bitmap"
⇒ association (長期關係)

Navigation

- 當Association關係中只有一方看得到另外一方，我們則用有箭頭的關聯線表達這樣的**可視性(navigation)**。



類量關係



⋮



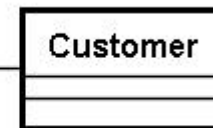
1..*



可以簡寫成這樣



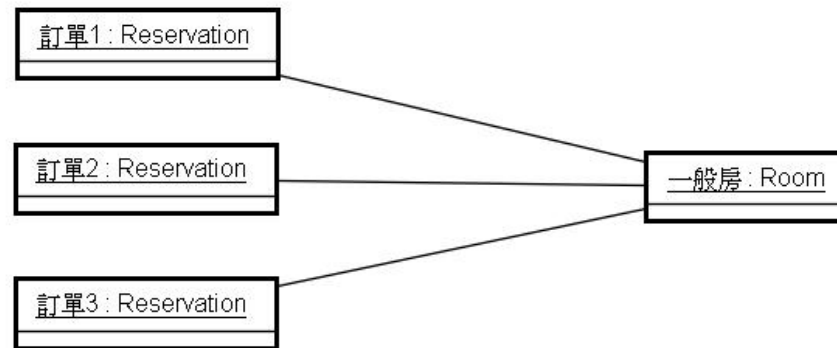
1



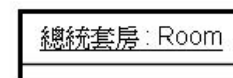
一個人可以訂好幾次房間

≥1

類量關係



有的房間有人訂，有的沒人訂

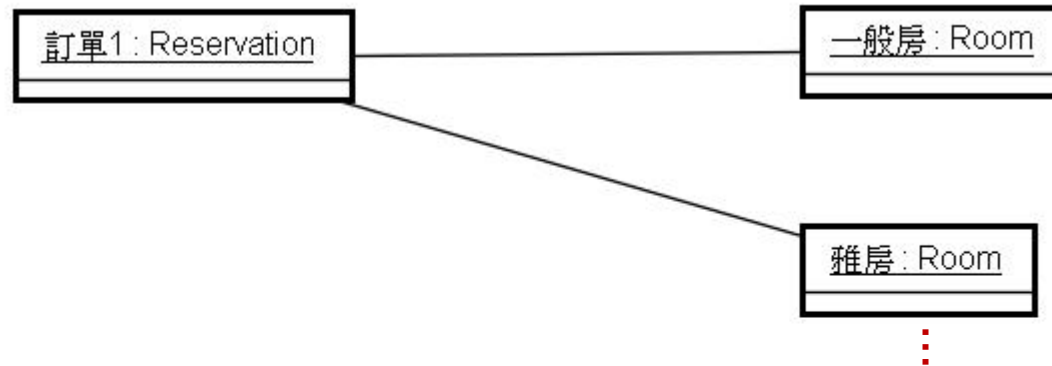


可以簡寫成這樣

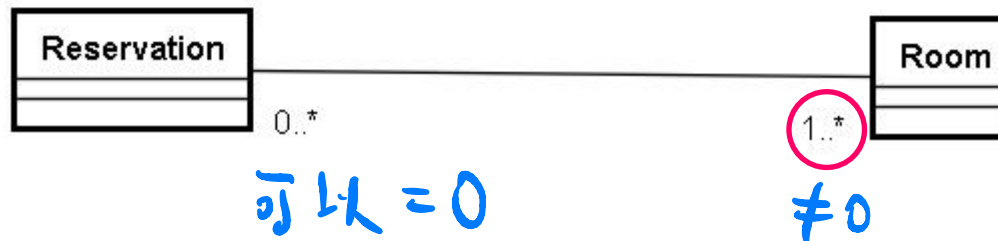


類量關係

一個訂單可訂好幾間房



可以簡寫成這樣

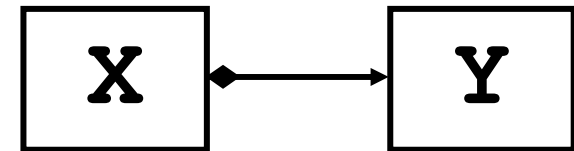


強聚合和弱聚合

- Both are “Has a” or “part-of” relationship

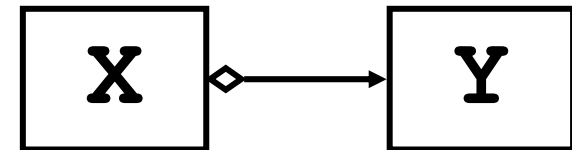
- Composition

- 強聚合
- 如果X不在，Y也會隨之消滅
- 身體vs 頭、四肢和驅幹



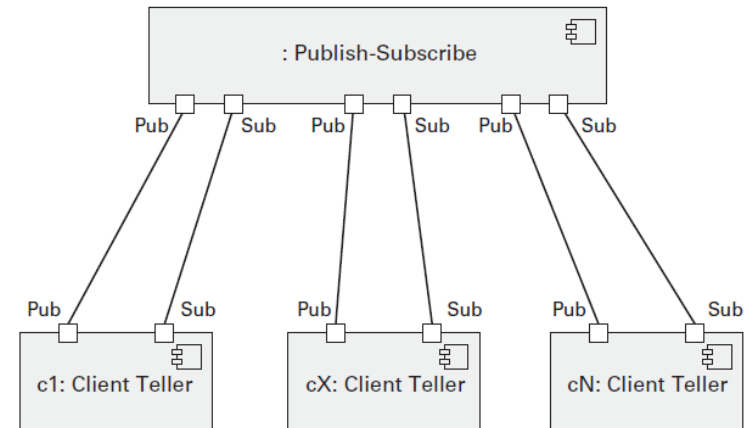
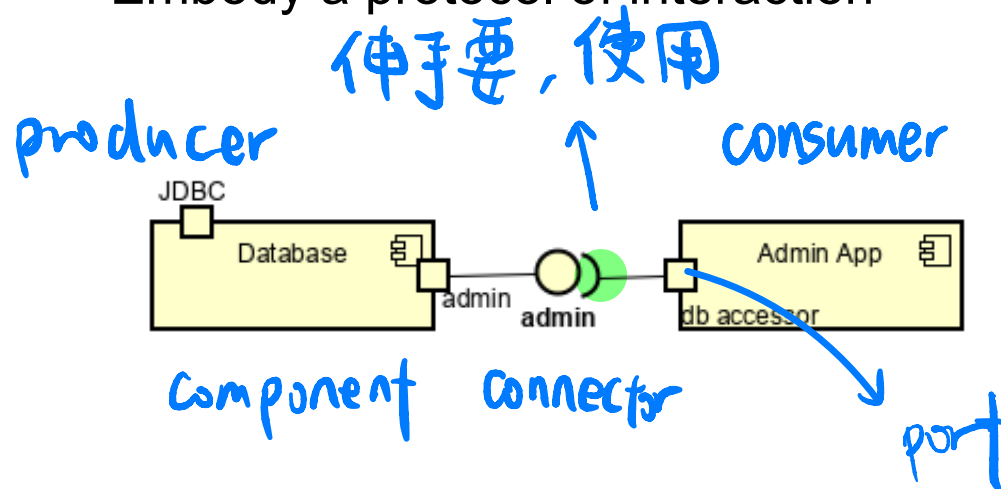
- Aggregation

- 弱聚合
- 如果X不在，Y仍然存在
- 飛機場vs飛機

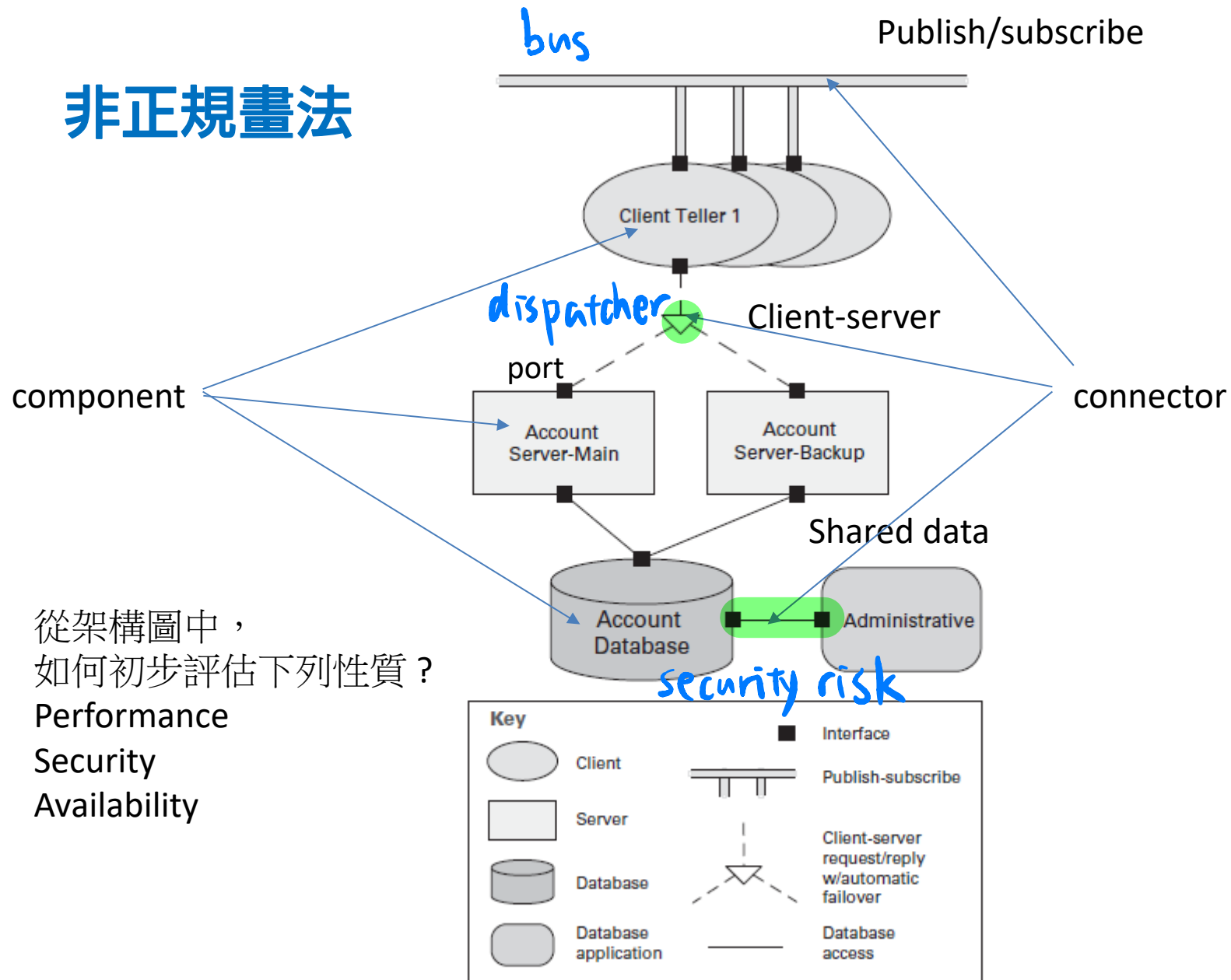


Documenting Distributed Structure

- Component
 - Processing units or data stores
 - Has a set of ports through which it interacts with other components via connectors
- Connector (連接的媒介)
 - Pathways between components
 - Embody a protocol of interaction

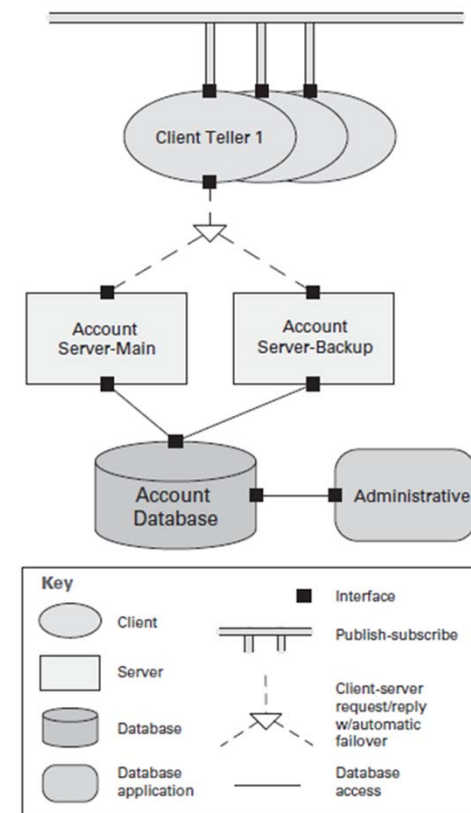


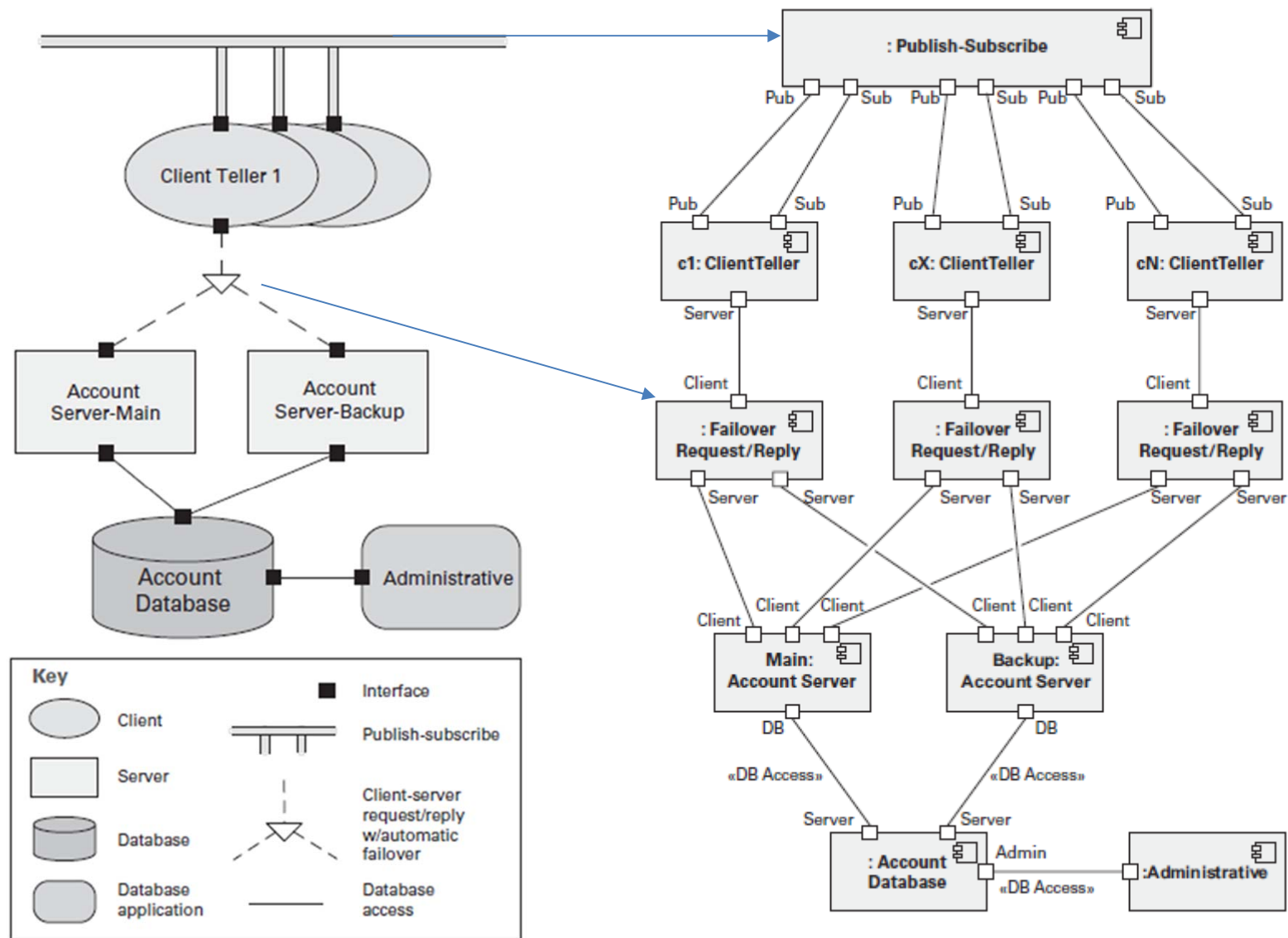
非正規畫法



Reasoning Quality Attribute in C&C connector & component

- Reliability
 - 元件/連線失效的可能性、造成的影響
- 效能
 - 特定元件的response time
 - 特定連線的latency
 - Buffer特性
- 資源需求
 - 特定元件需要多少硬體資源
- 安全
 - 那些元件需要限制存取權限
 - 那些連線需要加密



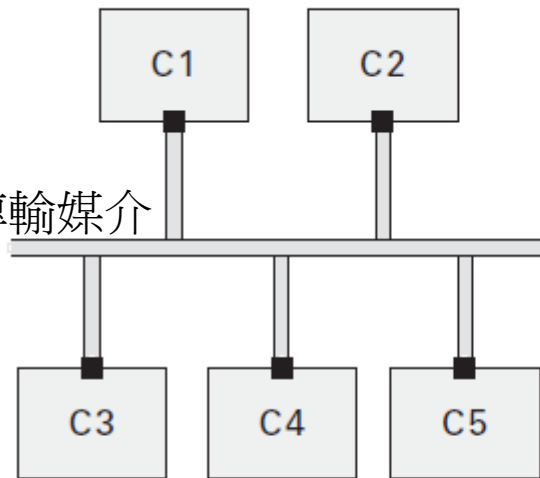


Implicit/Explicit Connector Notations

Implicit

Version 1

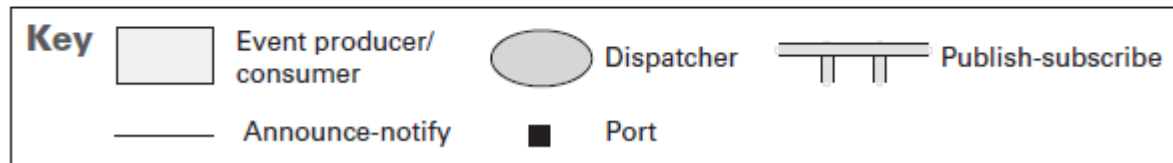
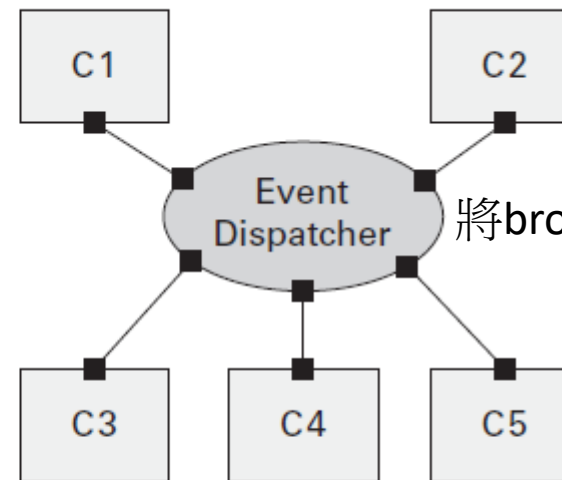
將broker做為傳輸媒介



Explicit

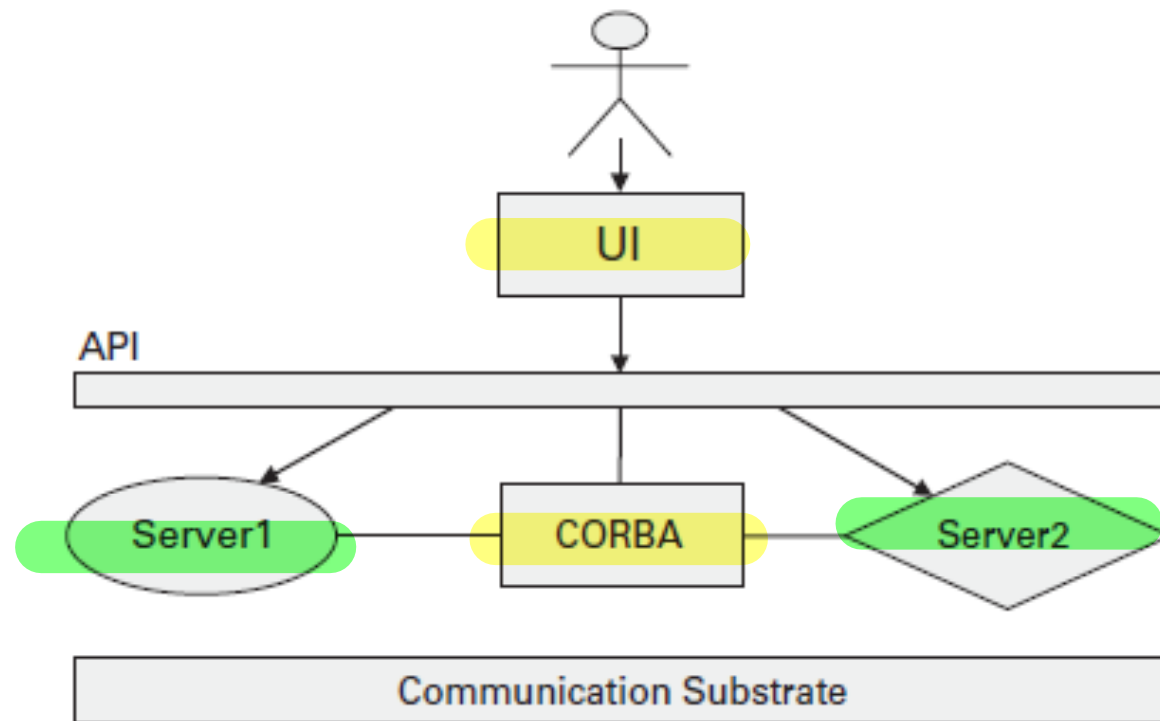
Version 2

將broker也做為元件

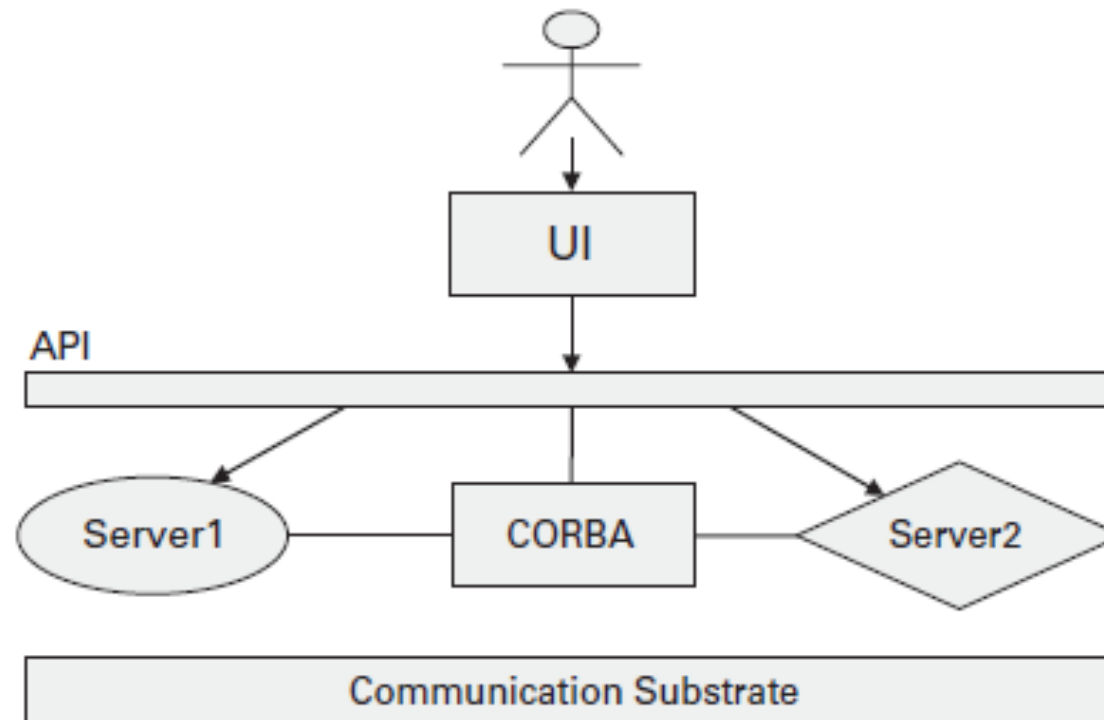


Case: 錯誤示範

這張圖那些地方不好？



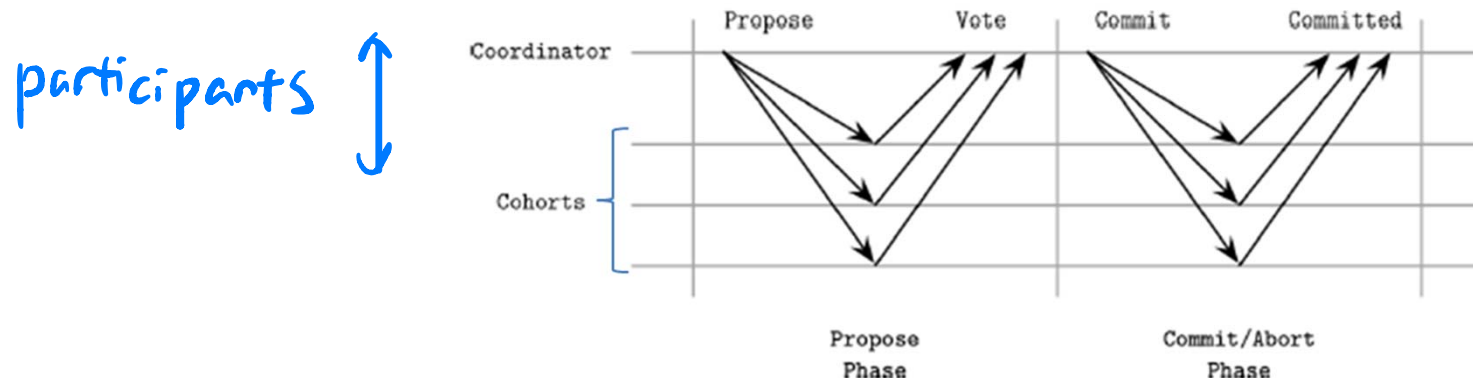
Legends ?



- It does not have a key;
- It uses different shapes for the same type of component;
- It uses the same shape for different types of components and connectors;
- It confuses the context with the system to be built;
- Its use of arrows is not explained;

Documenting Behaviors

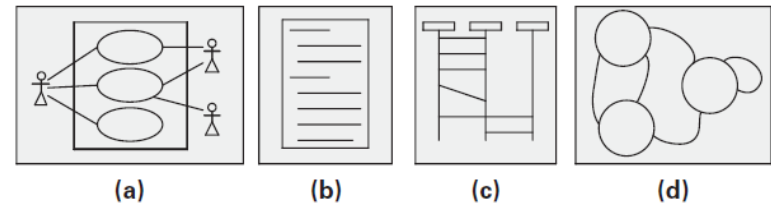
- What to document
 - Order*
 - Ordering of interactions among the elements
 - Opportunities for concurrency
 - Possible states of the system
 - Usage patterns for different system resources
 - memory, CPU, database connections, or network



→ *Time*

Documenting Behaviors

- To reveal
 - Stimulus and activity (外來刺激導致的系統行為)
 - **Ordering** of interactions (時間: 發生時序)
 - **Location** of the behavior (空間: 發生場域)
 - Elements or relations that the behavior maps to
- Two types of diagrams
 - Partial traces 只畫出: 眾多可能發生分支的其中一支
 - (b), (c) 用來描述細節
 - Comprehensive behaviors: 全貌
 - (a), (d) 用來提供概觀



動態系統描述 styles for behaviors

- 次序

- Use case diagram (a)

- 列出具系統運行代表性的scenario (use case diagram)

- Use case scenario (b, user story, partial trace)

- 針對每一個use case，寫出運作流程細節 開始 → 過程 → 結果

- Sequence diagram (c, partial trace)

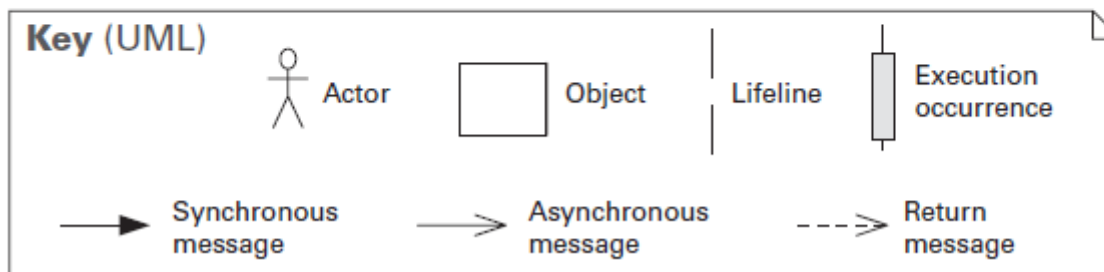
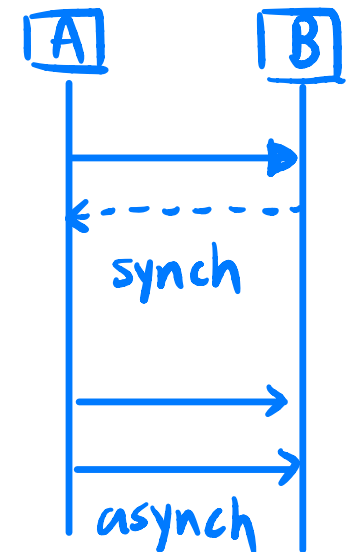
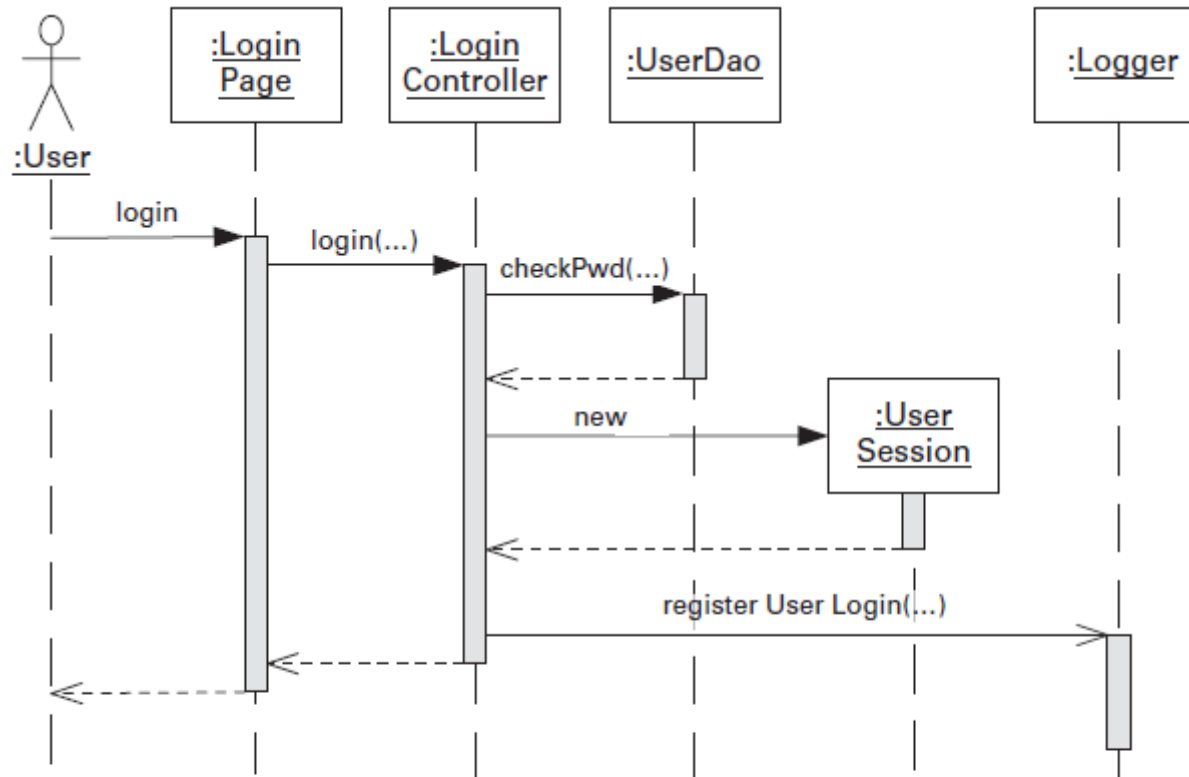
- 表述那些結構性元素如何互相合作來達成此scenario

- State diagram (d)

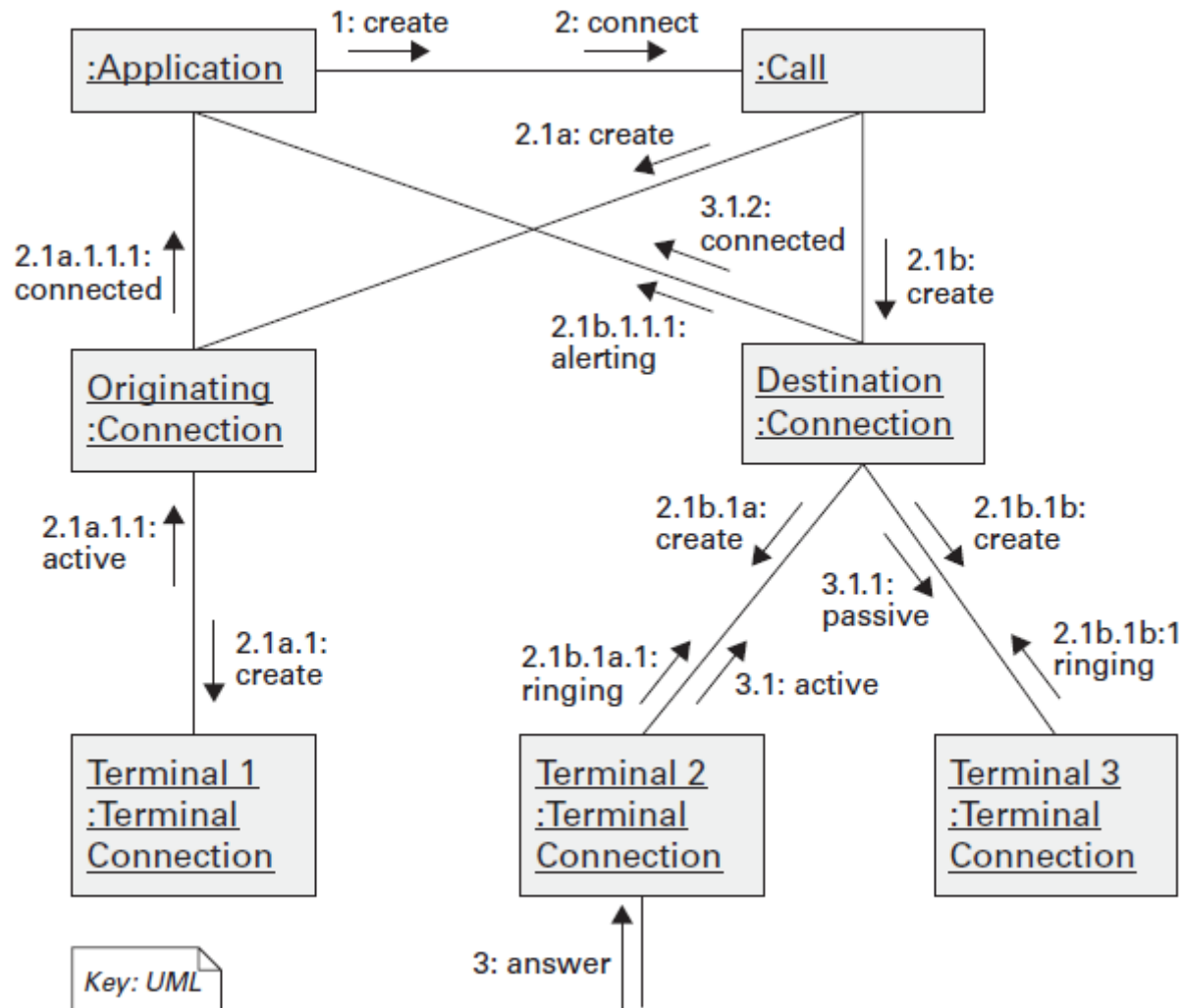
- 需要的話，描述結構性元素內部狀態的演變與觸發機制



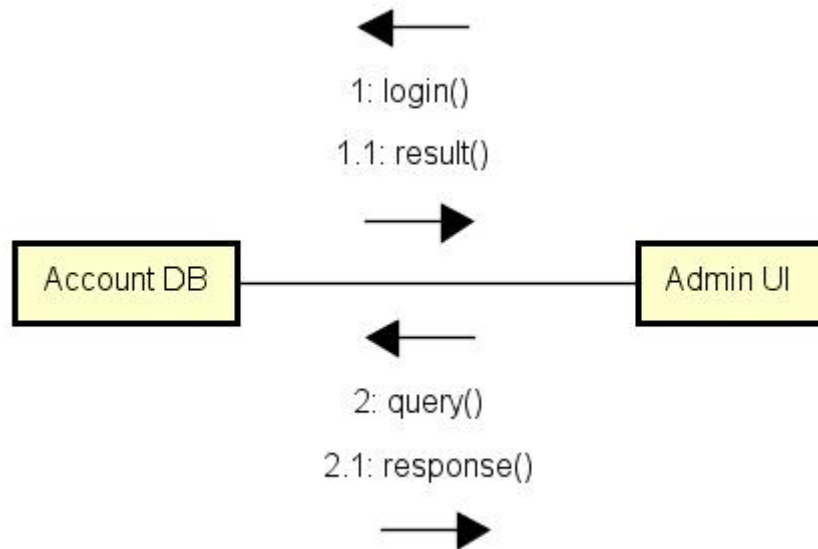
Sequence Diagram



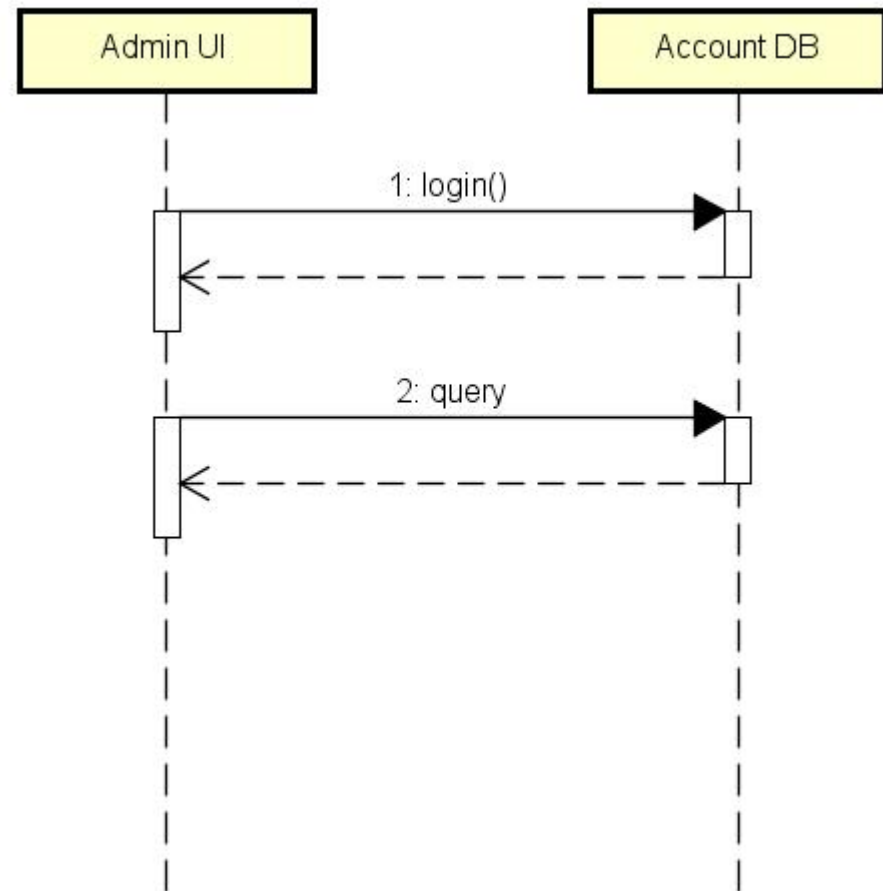
Communication Diagram



Seq. and Comm. Diagrams

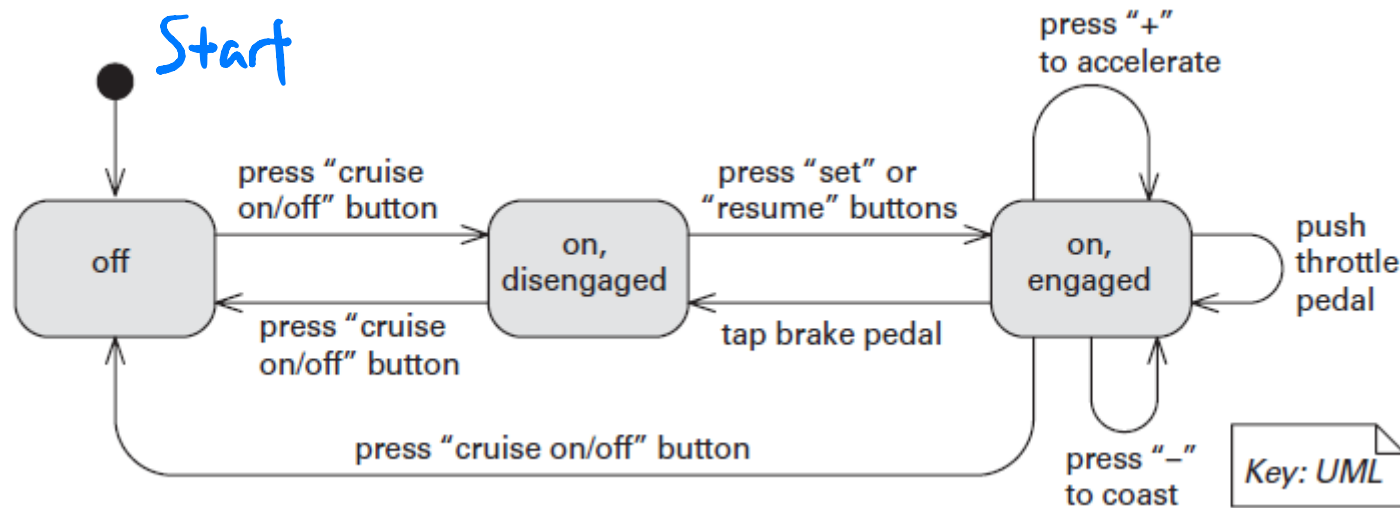


communication diagram



sequence diagram

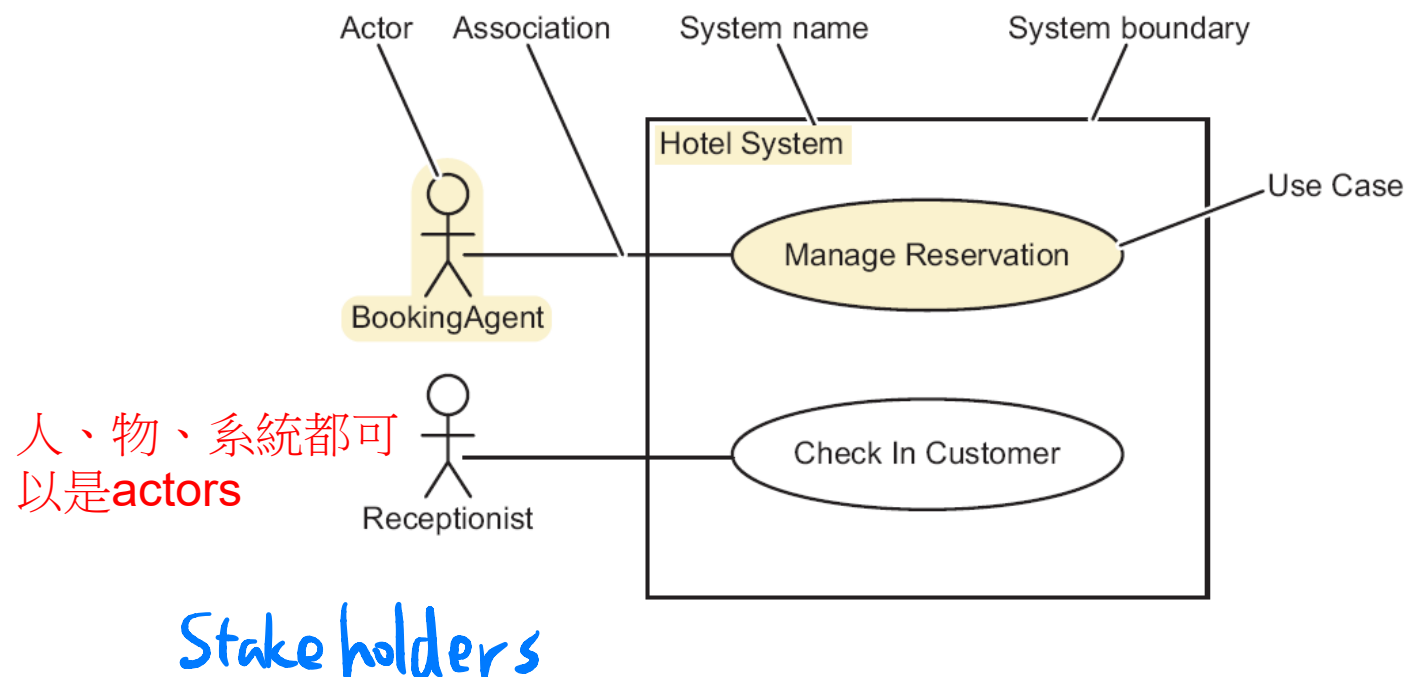
State Diagram



使用案例圖(Use Case Diagram)

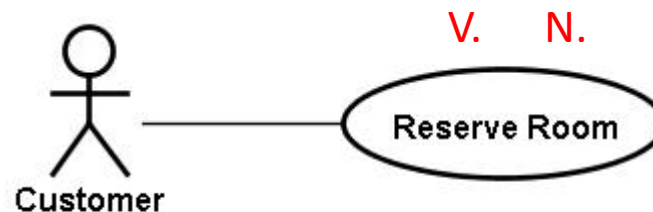
A use case diagram shows

- the relationships between actors (roles) and the system
- the goals that actors wish to achieve



Use Case (UC)

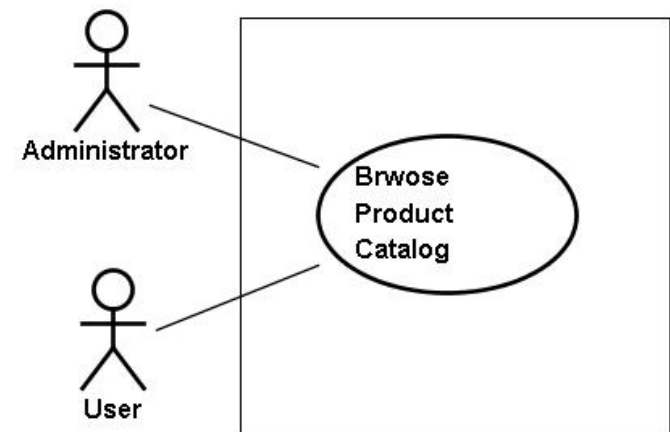
- Use Case
 - 描述系統目標
 - 目標: 一到多個Actors一連串與系統的互動，這些互動導致對該Actor來說「有價值」且「具體」的結果
 - 有價值: 符合Actor的「需求」
 - 具體: 系統輸出可以被明確描述 (definable outcome)
 - 夠具體才能被檢驗(測試)
 - 常犯錯誤: 描述UI



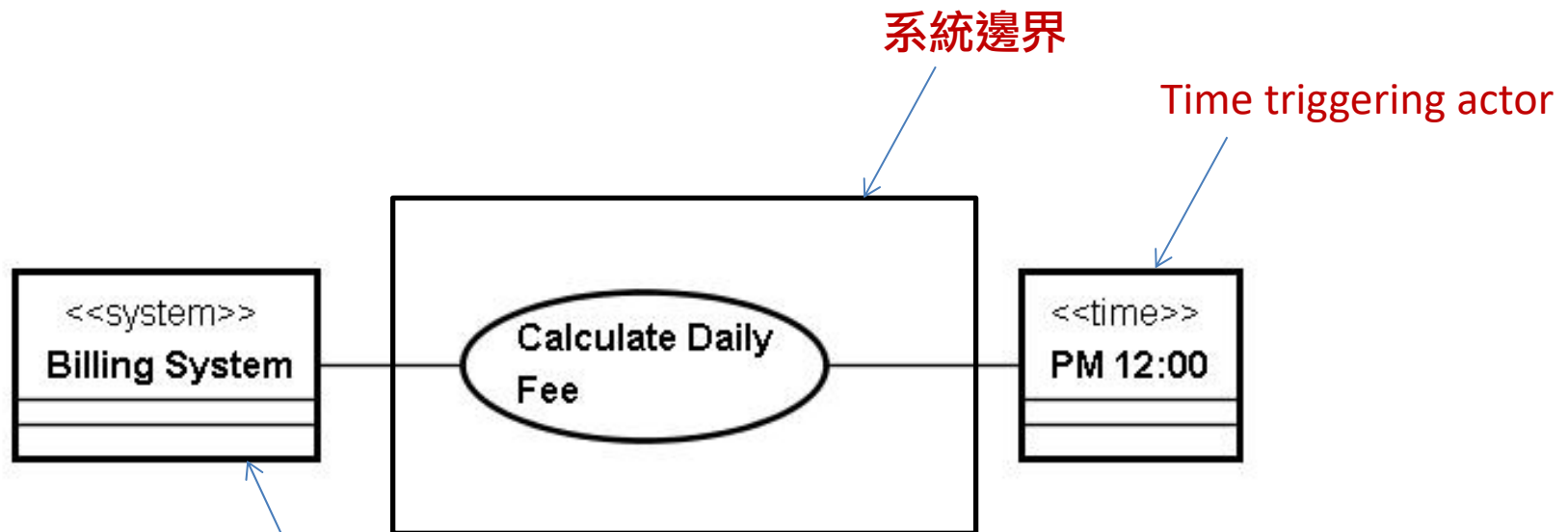
UC名字不宜過長，若過長可另於 Glossary 文件中定義

Actor

- 定義
 - 是一種角色(role)
 - 可以是人、外部系統、時間點(trigger)
 - 此角色與「要開發的系統」互動，完成UC
- 特色
 - 在「要開發的系統」外
 - 與「要開發的系統」互動
 - “連線”可代表全程或部份參與UC
 - 全程: Initiates or controls the UC
 - 部份: Participates parts of the UC



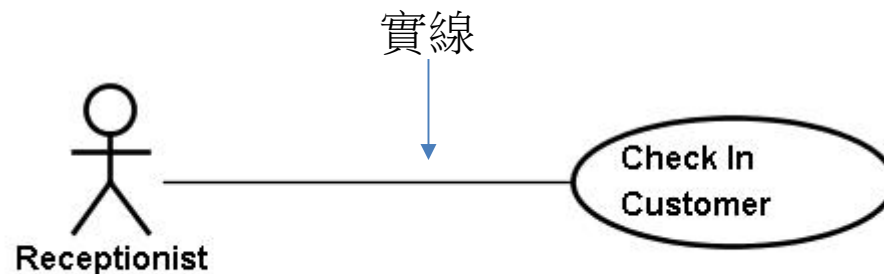
特別的Actors: Time與System



External system actor

這是一個外部系統，在「要開發的系統」範圍之外

關連(Association)



每個UC至少應該一個Actor關連

- 沒有連接Actor，就無法產生「價值」→不符定義

每個Actor至少也要和一個UC關連

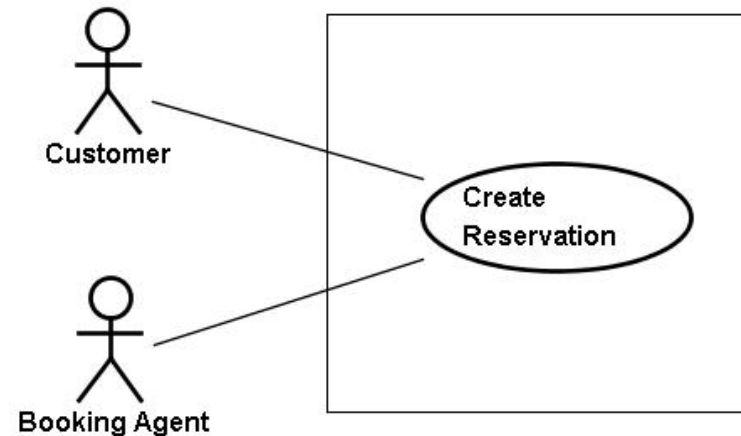
- 沒有連接UC，就無法完成UC→不符定義

UC: 一連串與系統的互動，這些互動導致對Actor來說產生「有價值」、「看得到」的結果

Actor: 一個與「要開發的系統」互動，完成UC的「角色」

使用案例情節 (Use Case Scenario)

- 使用案例情節描述下列行為
 - 參與者(actor)如與系統互動
 - 系統如何回應參與者
- 格式
 - The beginning *pre condition*
 - The middle
 - The end *post condition*



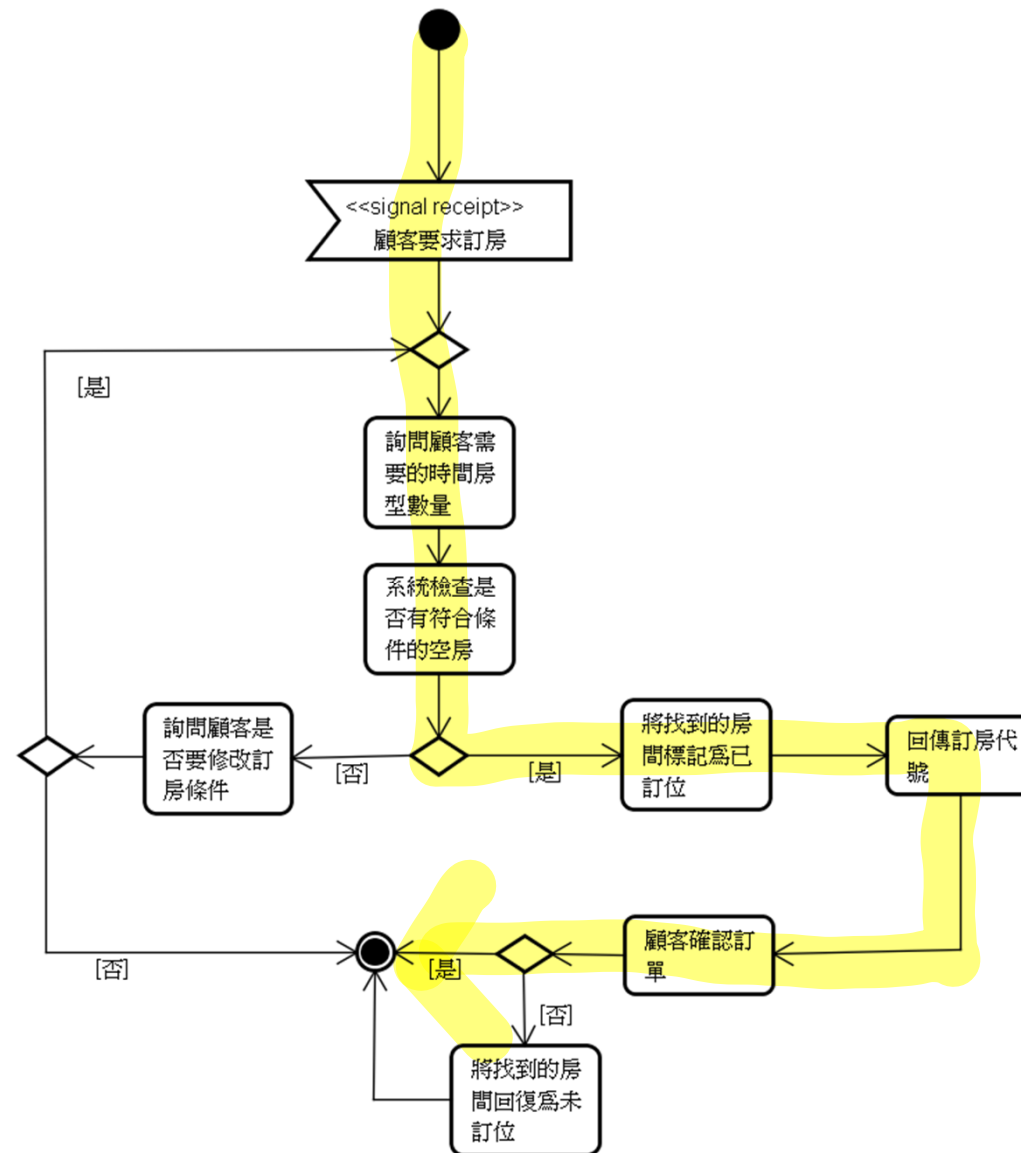
使用案例情節的寫作重點

- 當路徑很多時，不一定要寫作所有可能情節
 - 挑最重要的來寫
 - 看起來很複雜的
 - 系統執行時，可能很容易會有例外的
- Actor與系統的互動必須愈具體愈好

使用案例情節的寫作重點

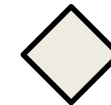
- 不要在一個案例中敘述多個條件分支
 - 具有不同結果的分支據情應該在寫在個別的使用案例情節中
 - 應該包含「成功(Happy path)」及「失敗(Sad path)」的情節
- 儘可能不要提及「使用者介面」及「實作技術」

Activity Diagram



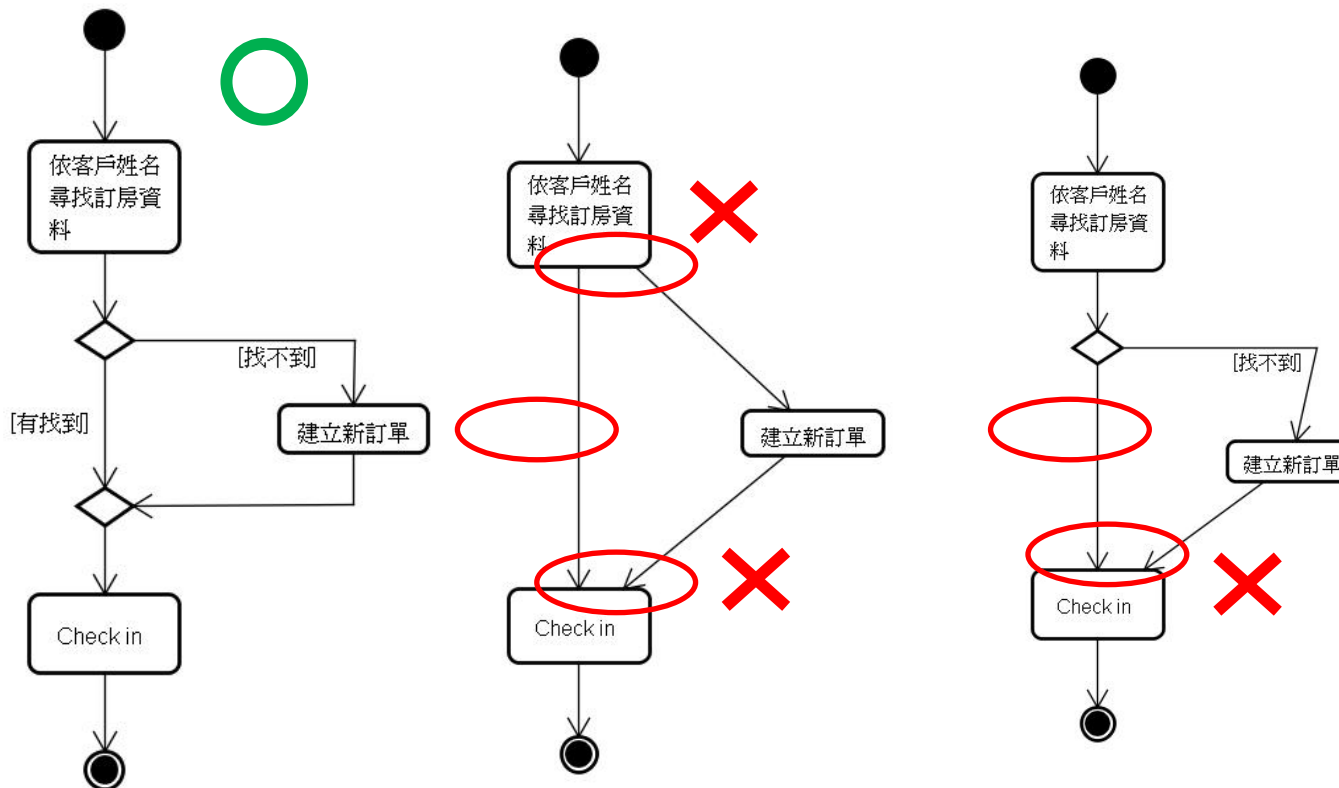
Activity Diagram

- Activity
 - 表示一個活動或一組活動
- Arrow
 - 標示執行的順序
- Start Node
 - 一組活動的開始
- Stop Node
 - 停止一個活動的所有控制流或物件流
- Branch Node
 - 表示一個決策條件



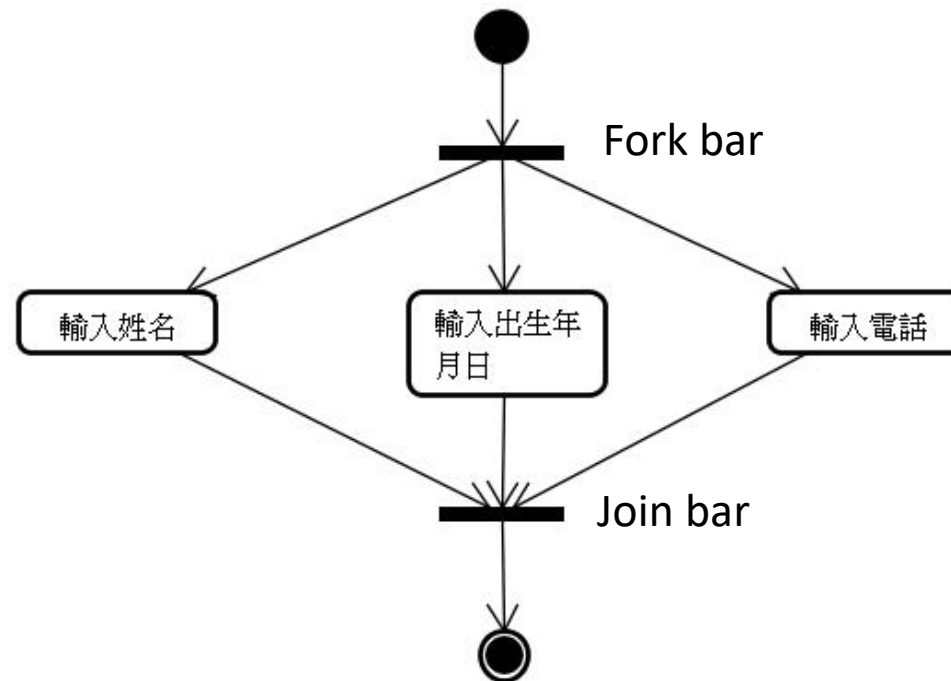
分支注意事項

- 一個Action只能有一出一入
 - 若需要分支、匯流，必須使用Branch/Merge
 - 分支一定要標明Guard

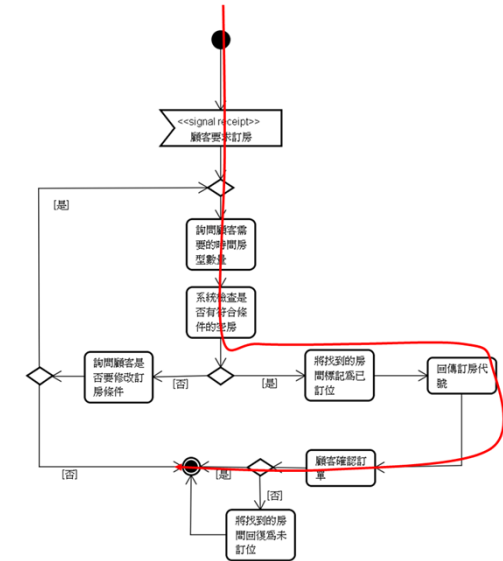


可同時進行的活動

- 通常用來表達不一定要依照先後次序，可同時進行的活動



使用案例情節



- The beginning
 - 這個使用案例開始於當訂房組人員接到顧客訂房的要求
- The middle

User如何操作系統

- 訂房組人員詢問顧客相關資訊，將顧客的「到達時間」「離開時間」「訂房數量及種類」輸入系統

系統如何回應

- 系統回應:有空房，它先將該房間標記為「已訂位」，回傳一個訂房代號

使用案例情節

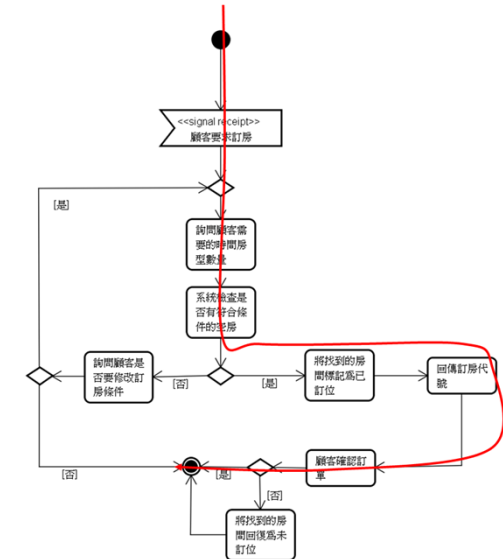
- The middle

User 如何操作系統

- 劣: 訂房組人員要求顧客確認訂單是否正確，顧客表示正確，訂房組人員將訂單狀態設定為成立
- 佳: 顧客確認訂單後，訂房組人員將訂單狀態設定為成立

系統如何回應

- 系統回應訂單狀態已成立

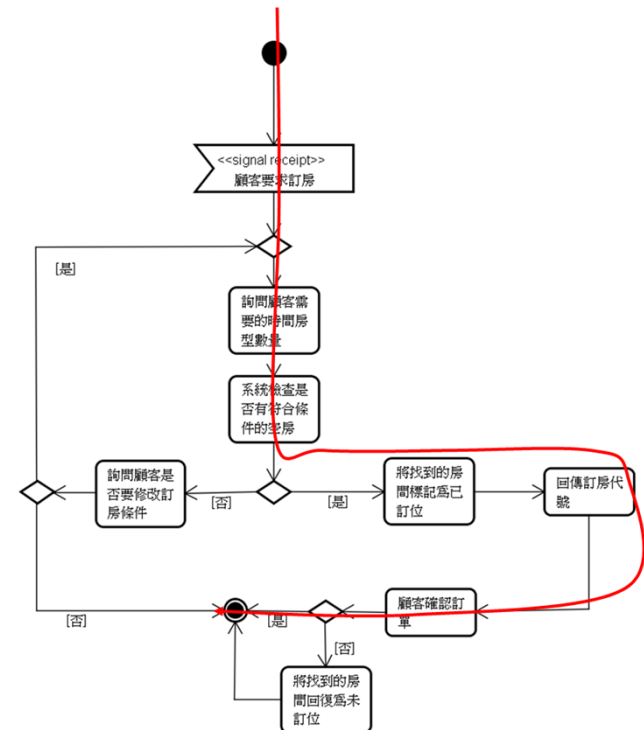


使用案例情節

- The end

描述系統最終狀態

- 最後，這位顧客所訂的房間被標記為「已訂」，且系統產生了一份報表，上面標名了「訂位代號」及相關細節



寫作軟體(架構)設計文件的原則

- 清楚明確
 - 名詞要有定義: 系統、框架、服務...
 - 行文簡潔不重覆
- 記錄重要的設計決定
 - 寫作「這樣設計的原因」
 - More why; less how
- 非標準圖示一定要有圖例
- 一次講一個面向，講清楚比講完整重要
 - 不同面向，分為不同圖與不同文件來說明

Recommended Reading

- Documenting Software Architectures: Views and Beyond
 - By Paul Clements, Felix Bachmann, Len Bass, David Garlan, James Ivers, Reed Little, Robert Nord, Judith Stafford

